TP de Biométrie — Semestre 5 Dynamique des Systèmes Écologiques

Benoît Simon-Bouhet Florian Orgeret

lundi 14 septembre 2026

Table des matières

Introduction **4**

Objectifs **5**

Pré-requis **7**

Organisation **8**

Contenu des séances **9**

Recommandation sur les outils d'assistance (IA) **11**

Évaluation(s) **12**
Licence 12

1 Structure des populations **13**
1.1 Objectifs 13
1.2 Avant de vous lancer… 13
1.3 Le Détective : Inférence et Probabilité de Détection (CMR) 14
1.3.1 Étape 1 : Regarder les données brutes 15
1.3.2 Étape 2 : L'ajustement du Modèle de Cormack-Jolly-Seber (CJS) 15
1.4 L'Architecte : Le Modèle Matriciel Structuré . . 17
1.4.1 Étape 1 : Décortiquer la matrice 19
1.4.2 Étape 2 : Cinétique et Projection Démographique 20
1.4.3 Étape 3 : Sensibilité et Spirale de l'Extinction . 21
1.5 À vous de jouer ! (Exercice autonome) 23

2 Analyse de cohortes **29**
2.1 Objectifs 29
2.2 Présentation de l'étude 29
2.3 Avant de vous lancer… 30
2.4 Les étapes de l'analyse 32
2.5 Sélection des données d'une date spécifique . . . 33
2.6 Structure démographique instantanée 34

2.7 Décomposition polymodale 37
2.7.1 Principe et étapes 37
2.7.2 Création du premier objet 38
2.7.3 Création du deuxième objet 40
2.7.4 Ajustement des lois Normales aux données . . . 42
2.7.5 Et en cas de message d'erreur ? 45
2.8 Taille moyenne et abondance des cohortes 46
2.9 Tableau et mise en forme des données 47
2.10 Courbe de croissance 49
2.11 Courbe de survie 50
2.12 Courbe d'Allen 51
2.13 Relation allométrique 52
2.14 À vous de jouer ! 53

3 Systèmes dynamiques 56
3.1 Objectifs 56
3.2 Pré-requis 56
3.3 Les boucles `for` 57
3.4 Modèle de mortalité exponentielle 60
3.5 Modèle de croissance exponentielle 62
3.6 Modèle de croissance logistique 64
3.7 Modèle prédateur-proie de Lotka-Volterra 66
3.8 Modèle de compétition interspécifique 70
3.9 Modèles à 3 espèces 71

4 Les Chaînes de Markov 74
4.1 Pré-requis 74
4.2 Les matrices dans `R` 75
4.2.1 Création 75
4.2.2 Produit matriciel 78
4.3 Simulation d'écosystème 83

Références 86

Introduction

Objectifs

Ce document rassemble les supports et consignes pour les 4 séances de travaux pratiques dédiées à la **dynamique des populations et des systèmes écologiques**.

Bien qu'administrativement rattachés à l'**UE Biométrie** (Semestre 5, Licence Sciences de la Vie de La Rochelle Université), ces TP s'articulent directement avec les enseignements magistraux du module **« Introduction à la dynamique des systèmes »** au sein de l'EC *Fonctionnement des Écosystèmes*.

Les cours magistraux ainsi que l'ensemble de ces 4 séances de TP sont assurés par **Florian Orgeret**.

À travers ces travaux pratiques, vous mobiliserez le logiciel `R` comme un outil de simulation numérique pour modéliser, visualiser et assimiler concrètement les processus écologiques abordés en cours :

À travers ces 4 travaux pratiques, vous apprendrez à mobiliser `R` pour modéliser et comprendre les processus démographiques :

Chapitre 1

- Estimer des paramètres démographiques (taux de survie, probabilités de recapture) à partir de données CMR (modèles CJS avec `marked`).
- Modéliser et projeter la dynamique de populations structurées via des matrices de Leslie et de Lefkovitch (`popbio`).

Chapitre 2

- Décomposer des distributions de tailles polymodales (méthodes de Bhattacharya / Hasselblad via `mixdist`).
- Ajuster des modèles de croissance individuelle et bâtir des courbes de survie et de production (courbes d'Allen).

Chapitre 3

— Simuler le comportement et analyser la sensibilité de modèles continus et discrets (croissance logistique).
— Modéliser les interactions biotiques : dynamique prédateurs-proies de Lotka-Volterra et compétition interspécifique.

Chapitre 4

— Modéliser les probabilités de transition entre états ou communautés écologiques.
— Simuler les dynamiques spatio-temporelles et calculer la distribution stationnaire (état d'équilibre) du système.

Pré-requis

Domaine	Outils clés	Compétences attendues
Gestion de projet	`RStudio`	Créer un projet, structurer un script reproductible
Manipulation	`dplyr`, `tidyr`	Filtrer, transformer, agréger (`filter`, `mutate`, `summarise`)
Visualisation	`ggplot2`	Construire des figures lisibles et adaptées au type de variable

⚠ Si ces pré-requis ne sont pas maîtrisés

En cas de lacunes sur la syntaxe de base, reportez-vous aux supports précédents : :

— le livre en ligne de Biométrie du semestre 3

— le livre en ligne de Biométrie du semestre 4

Vous aurez de toutes façons besoin de vous référer à ces 2 documents régulièrement au cours de ces TP. Donc gardez-les à portée de main et apprenez à naviguer rapidement dans leurs chapitres et sous-chapitres pour retrouver facilement les informations dont vous avez besoin.

Organisation

Présence obligatoire : Ces 4 séances de modélisation appliquée nécessitent un accompagnement pas à pas ; la présence en salle informatique est requise.

Environnement de travail : Vous pouvez utiliser les postes informatiques de l'université ou votre propre ordinateur portable (Windows, macOS, Linux). Assurez-vous d'avoir installé les packages listés au début de ce document (`tidyverse`, `popbio`, `mixdist`, `marked`).

Contenu des séances

1. **Séance consacrée à la structure et à la démographie des populations** (Chapitre 1). Vous aborderez deux approches fondamentales pour quantifier et projeter la dynamique d'une population structurée. Dans un premier temps, vous utiliserez des données de suivi individuel (capture-marquage-recapture) sur le cincle plongeur pour estimer, via les modèles de Cormack-Jolly-Seber (CJS) avec le package marked, les taux de survie apparente et les probabilités de recapture en tenant compte d'effets temporels ou de covariables. Dans un second temps, vous construirez des matrices de transition démographique (matrices de Leslie et de Lefkovitch) à l'aide du package popbio. Vous projetterez la dynamique d'espèces à cycle de vie contrasté (baleine franche, tortue de Floride), calculerez le taux intrinsèque d'accroissement (λ), la structure d'âge stable, ainsi que les sensibilités et élasticités des différents paramètres du cycle de vie. L'objectif est de vous apprendre à lier l'estimation empirique de paramètres démographiques de terrain à la modélisation prédictive matricielle pour éclairer la gestion et la conservation des populations.
2. **Séance consacrée à l'analyse de cohortes** (Chapitre 2). Avant votre sixième séance de TP, vous devriez être en mesure de réaliser l'analyse de cohorte d'une population étudiée pendant plusieurs années afin de produire les courbes de croissance, de survie et d'Allen d'une cohorte d'intérêt. Vous devrez en particulier importer et mettre en forme des données issues d'un suivi de terrain, produire les structures démographiques instantanées à chaque date d'échantillonnage, faire les décompositions polymodales afin de récupérer les informations utiles au sujet de la cohorte dont on souhaite assurer le suivi et enfin, produire, pour cette cohorte, les courbes de croissance, de mortalité et d'Allen.

3. **Séance consacrée à l'étude des systèmes dynamiques** (Chapitre 3). Vous devrez coder différents modèles d'évolution d'une population (mortalité exponentielle, croissance exponentielle, croissance logistique) ou de plusieurs populations ou espèces (modèle prédateurs-proies, modèle de compétition). Ces modèles généreront des données que vous devrez représenter graphiquement. Vous devrez enfin modifier la valeur de certains paramètres de ces modèles pour comprendre leur influence sur le comportement des systèmes dynamiques étudiés. Pour la première fois, vous utiliserez donc `RStudio` pour produire des données qui suivent un modèle que vous spécifierez, au lieu d'importer des données déjà disponibles. L'objectif est ici de vous montrer comment utiliser le logiciel pour étudier le comportement de modèles théoriques de dynamique des populations, et de vous permettre d'évaluer l'importance et la sensibilité des conditions initiales et des valeurs des paramètres des modèles.
4. **Séance est consacrée à l'utilisation des chaînes de Markov dans un contexte d'écologie des communautés** (Chapitre 4). Vous verrez comment utiliser `RStudio` pour coder une chaîne de Markov permettant d'étudier les successions écologiques et l'atteinte de l'état d'équilibre d'un écosystème abritant plusieurs communautés végétales. Comme pour la semaine précédente, vous devrez coder le système, représenter graphiquement les résultats de simulation, et examiner les conséquences de changements des conditions initiales ou des probabilités de transitions entre communautés. Comme toujours, l'objectif est donc double : vous permettre de découvrir de nouvelles possibilités d'analyses réalisables avec `R` et `RStudio`, et vous permettre de mieux comprendre les notions fondamentales abordées dans les cours magistraux.

Recommandation sur les outils d'assistance (IA)

i L'IA comme levier d'apprentissage, non comme raccourci

Les modèles d'IA générative (ChatGPT, Claude, etc.) sont d'excellents alliés pour **dépasser la frustration technique**. N'hésitez pas à les solliciter lorsque votre code bloque : ils sont particulièrement efficaces pour expliciter un message d'erreur de la console, débusquer une coquille de syntaxe ou vous aider à corriger un bug. Utilisée comme un tuteur pour débloquer la programmation, l'IA fait gagner un temps précieux et soutient votre progression.
Gardez toutefois à l'esprit la finalité de ces simulations : **construire pour comprendre**. Obtenir un code fonctionnel en copiant l'énoncé dans une IA est désormais immédiat, mais court-circuiter la réflexion se fait au détriment de l'assimilation des concepts écologiques, qui demande un véritable effort d'apprentissage sur le temps long.
Utilisez donc ces outils pour comprendre vos erreurs et réparer votre code, jamais pour vous dispenser de réfléchir aux mécanismes biologiques modélisés.

Évaluation(s)

Il n'y aura pas d'évaluation notée directement sur ces séances de TP, ni de compte rendu technique ou de script R à rendre.

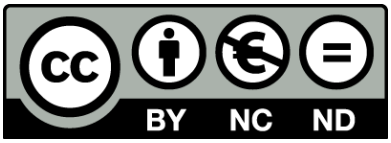
L'examen terminal portera non pas sur la maîtrise du code R ou sur la technicité de la programmation, mais exclusivement sur la **compréhension approfondie des concepts d'écologie des populations et de dynamique des écosystèmes** illustrés durant ces séances.

L'ordinateur et le logiciel `R` ne sont ici que des instruments pédagogiques : manipuler les modèles, faire varier les paramètres et observer directement les trajectoires démographiques appliquent la logique du « **construire pour comprendre** ».

Prendre le temps de suivre activement chaque séance, faire l'effort de manipuler les modèles par vous-mêmes, refaire les exercices et étudier attentivement les corrigés constituera donc un avantage certain pour réussir l'examen théorique.

Licence

Ce support est mis à disposition selon les termes de la licence Creative Commons CC BY-NC-ND 4.0.



Attribution — Pas d'utilisation commerciale — Pas de modification sans accord préalable des auteurs.

1 Structure des populations

1.1 Objectifs

Ce TP fait le pont direct avec vos cours magistraux. Vous allez y explorer les deux facettes fondamentales de l'écologie quantitative :

1. **L'approche du détective (Modèle statistique) :** Comment estimer la vraie survie d'une population quand on rate des individus sur le terrain (détection imparfaite) ?
2. **L'approche de l'architecte (Modèle de processus) :** Comment utiliser ces estimations pour construire un système, simuler l'avenir et tester des stratégies de conservation ?

À l'issue de cette séance, vous serez capables de :

— Lire et interpréter un historique de Capture-Marquage-Recapture (CMR).
— Ajuster un modèle simple pour séparer la survie (ϕ) de la probabilité de détection (p).
— Construire une matrice de Leslie et simuler la dynamique d'une population structurée en stades (cinétique).
— Évaluer la sensibilité d'une espèce longévive, en lien avec la "spirale de l'extinction" vue en cours.

1.2 Avant de vous lancer...

Pour travailler sereinement, créez un nouveau script (nommez-le par exemple `TP1_Systemes_Dynamiques.R`) à l'intérieur de votre projet sous RStudio.

Bonne nouvelle : **vous n'aurez aucun fichier externe à télécharger ou à importer pour ce TP !** Les données sont déjà intégrées dans les packages R que nous allons utiliser.

Vous avez juste besoin de charger trois bibliothèques (installez-les via la console de RStudio si ce n'est pas encore fait avec `install.packages(c("marked", "popbio", "tidyverse"))`) :

```
library(tidyverse) # La caisse à outils pour manipuler les données
library(marked)    # Pour les modèles de Capture-Marquage-Recapture (Le Détective)
library(popbio)    # Pour les matrices de dynamique des populations (L'Architecte)
```

1.3 Le Détective : Inférence et Probabilité de Détection (CMR)

Sur le terrain, la réalité est complexe : un animal vivant n'est pas toujours observé. Comme vu en CM avec les comptages de mésanges en forêt, la probabilité de détection (p) est presque toujours inférieure à 1.

Nous allons utiliser le jeu de données `dipper` (inclus dans le package `marked`). Il s'agit d'un suivi réel du Cincle plongeur (*Cinclus cinclus*), un petit oiseau aquatique, suivi pendant plusieurs années.



FIGURE 1.1 : Le Cincle plongeur (*Cinclus cinclus*), inféodé aux cours d'eau rapides et suivi par baguage (Source : Wikimedia Commons, CC-BY-SA).

1.3.1 Étape 1 : Regarder les données brutes

Avant de modéliser, regardons à quoi ressemble un fichier CMR.

```
# Chargement et affichage des 6 premières lignes du jeu de données
data(dipper)
head(dipper)
```

```
      ch    sex
1 0000001 Female
2 0000001 Female
3 0000001 Female
4 0000001 Female
5 0000001 Female
6 0000001 Female
```

Observation : Dans la colonne `ch` (*capture history*), vous voyez des suites de 0 et de 1 (ex : `1100000`).

- Un `1` signifie que l'oiseau a été capturé ou revu cette année-là.
- Un `0` signifie qu'il n'a **pas** été vu.

Si vous voyez un historique comme `1010000`, cela signifie que l'oiseau a été vu l'année 1, raté l'année 2, puis revu l'année 3. **Puisqu'il a été revu l'année 3, il était obligatoirement vivant l'année 2 !** Cet historique prouve que l'on rate des individus et justifie l'utilisation de modèles mathématiques.

1.3.2 Étape 2 : L'ajustement du Modèle de Cormack-Jolly-Seber (CJS)

Nous allons demander à R d'estimer la survie (Φ) et la probabilité de détection (p). Nous utilisons l'argument `formula = ~1` pour dire à R : *"Estime une valeur moyenne constante pour toute la période, sans différencier les années ou les sexes"*.

```r
# Ajustement du modèle CJS
modele_cjs <- crm(dipper,
                  model = "cjs",
                  model.parameters = list(Phi = list(formula = ~1),
                                          p = list(formula = ~1)))

# Affichage des probabilités réelles estimées
predict(modele_cjs)
```

```
$Phi
  occ  estimate
1   6 0.5602139

$p
  occ  estimate
1   7 0.9026536
```

> **Point R :** Ne vous laissez pas impressionner par les lignes de calcul de l'algorithme d'optimisation dans la console de RStudio. Regardez le tableau final produit par `predict()`.
>
> — La probabilité de survie moyenne d'une année sur l'autre (Φ) est d'environ **0.56 (56%)**.
> — La probabilité de détection (p) est d'environ **0.90 (90%)**. Cela signifie que les chercheurs ont raté en moyenne 10% des oiseaux vivants à chaque sortie.

Une valeur moyenne globale masque parfois des disparités biologiques fondamentales. En écologie des populations, les rôles écologiques, l'effort reproducteur ou les comportements territoriaux diffèrent souvent entre mâles et femelles. Vérifions si la survie (Φ) est équivalente entre les sexes en intégrant la covariable `sex` dans les formules du modèle :

```r
mod_sexe <- crm(dipper, model = "cjs",
                model.parameters = list(Phi = list(formula = ~sex),
                                        p   = list(formula = ~1)))
```

Utilisez ensuite la fonction `predict` pour afficher l'estimation des paramètres du modèle :

```
predict(mod_sexe)
```

```
$Phi
     sex occ  estimate
1 Female   6 0.5507352
2   Male   6 0.5702637

$p
  occ  estimate
1   7 0.9026907
```

Questions d'interprétation :

1. **Survie différentielle et probabilités cumulées :**
   — Quelle est la différence annuelle brute de survie entre mâles et femelles ?
   — Calculez la probabilité pour un individu d'être encore vivant au bout de 5 ans dans chaque sexe (Φ^5). Que constatez-vous sur l'écart relatif entre mâles et femelles après quelques années ?
2. **Biais de détection (p) :**
   — Si un écologue de terrain n'utilisait pas de modèle CJS et calculait simplement le ratio des oiseaux revus d'une année sur l'autre, quelle valeur approchée de survie obtiendrait-il ($\Phi \times p$) ? Pourquoi confondre « non-détecté » et « mort » est-il risqué ?
   — Comparez la probabilité de détection (p) estimée pour les femelles et pour les mâles (changer le code pour vous-même). Constatez-vous une différence notable ? En vous renseignant brièvement sur l'aspect visuel (plumage) et les comportements territoriaux du Cincle plongeur (*Cinclus cinclus*), quelles caractéristiques biologiques de cette espèce expliquent logiquement cette divergence (ou similitude) de détectabilité pour l'observateur ?

1.4 L'Architecte : Le Modèle Matriciel Structuré

Le détective vient d'estimer la survie (Φ) et la détection (p). Mais la survie n'est qu'une pièce du puzzle.

Sur le terrain, les écologues déploient une vaste panoplie de modèles statistiques (que nous ne verrons pas dans ce TP) pour estimer l'ensemble des traits d'histoire de vie d'une population, par exemple :

- **La fécondité (F) :** estimée par le suivi des pontes, des nichées ou par des modèles de comptage (ex. modèles de Poisson ou binomial négatif).
- **L'âge à la première reproduction (E) :** estimé via des modèles multi-états ou des suivis longitudinaux d'individus d'âge connu.
- **Les probabilités de transition / maturation (ψ) :** estimées via des modèles de Capture-Marquage-Recapture multi-sites ou multi-états (ex. passage du statut "juvénile" à "adulte").

Une fois ces paramètres estimés par les « détectives », ils ne restent pas de simples chiffres isolés dans un tableau : **l'architecte s'en empare pour construire la matrice de projection**.

Chaque case d'une matrice démographique (Matrice de Leslie ou de Lefkovitch) correspond précisément à l'un de ces paramètres vitaux estimés sur le terrain. Nous changeons maintenant d'espèce et d'échelle : nous allons assembler ces briques pour simuler le devenir de la population et orienter les décisions de gestion.

Nous allons analyser la population d'Orques (*Orcinus orca*), avec le jeu de données `whale` du package `popbio`.



FIGURE 1.2 : Orque mâle adulte (*Orcinus orca*). Reconnaissable à son imposant aileron dorsal triangulaire pouvant atteindre jusqu'à 2 mètres de hauteur. Même adultes, les mâles restent généralement fidèles à leur groupe familial matrilinéaire, guidé par les femelles âgées (Source : Wikimedia Commons).

1.4.1 Étape 1 : Décortiquer la matrice

```
# Chargement et affichage de la matrice de l'orque
data(whale)
whale
```

```
           yearling juvenile mature postreprod
yearling     0.0000   0.0043 0.1132     0.0000
juvenile     0.9775   0.9111 0.0000     0.0000
mature       0.0000   0.0736 0.9534     0.0000
postreprod   0.0000   0.0000 0.0452     0.9804
```

Observation : Cette matrice carrée 4×4 répartit les individus en 4 stades de développement : *yearling* (1 an), juvénile, adulte mature, et post-reproductive (ménopause).

— Les colonnes représentent le temps t.
— Les lignes représentent le temps $t + 1$.
— La **première ligne** contient la fécondité (seules les adultes matures se reproduisent, avec un taux de 0.04).
— Les **autres lignes** contiennent les probabilités de survie et de passage d'un stade à l'autre.

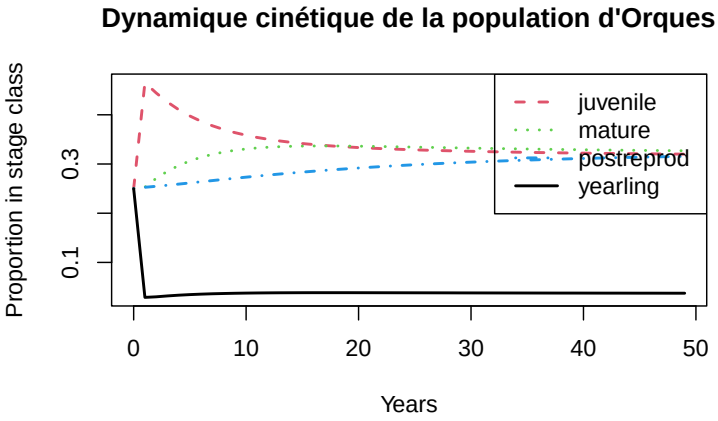
1.4.2 Étape 2 : Cinétique et Projection Démographique

La cinétique décrit “comment” l’abondance varie dans le temps. Simulons ce qui se passerait sur 50 ans si nous démarrions avec une toute petite population de 40 orques (10 dans chaque stade).

```
# 1. On crée notre vecteur de population de départ
n_initial <- c(10, 10, 10, 10)

# 2. On lance la projection mathématique sur 50 ans
projection_orque <- pop.projection(A = whale, n = n_initial, iterations = 50)

# 3. On visualise les résultats
stage.vector.plot(projection_orque$stage.vectors,
                  col = 1:4,
                  main = "Dynamique cinétique de la population d'Orques")
```



Observation : Regardez les 10 premières années sur le graphique. Les courbes zigzaguent ! C’est ce qu’on appelle **l’inertie démographique** vue en CM. La population met du temps à se “rééquilibrer” par rapport à notre structure de départ artificielle. Ensuite, les courbes deviennent parallèles : la population a atteint sa distribution stable.

La population va-t-elle s’éteindre ou croître ? Pour le savoir, on calcule λ (le taux de croissance asymptotique).

```r
lambda(whale)
```

```
[1] 1.025441
```

Si $\lambda > 1$, la population croît. S'il est < 1, elle décline. Ici, $\lambda \approx 1.025$, soit une croissance très lente de 2.5% par an, typique des grands mammifères.

1.4.3 Étape 3 : Sensibilité et Spirale de l'Extinction

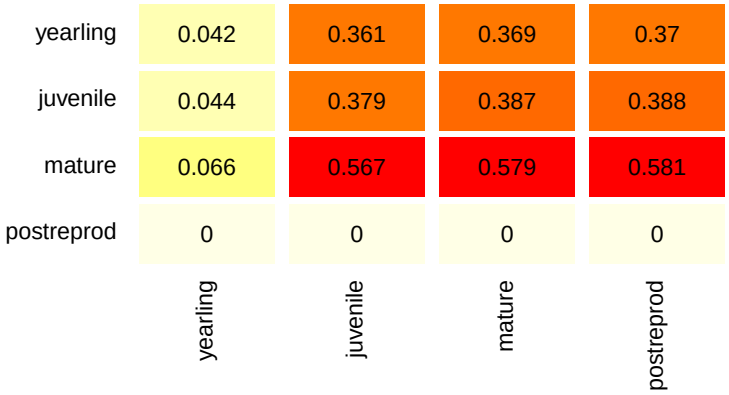
L'orque est une espèce à vie longue (longévive). En cours, nous avons vu que ces espèces sont extrêmement sensibles à une baisse de la survie des adultes. Vérifions-le avec la fonction `sensitivity()`.

```r
# Calcul de la matrice de sensibilité
sensibilite_orque <- sensitivity(whale)

# Représentation graphique :
image2(sensibilite_orque,
       log = FALSE,
       col = c("white", rev(heat.colors(23))),
       labels = c(1, 2),
       mar = c(4.5, 5, 3.5, 1),
       box.offset = 0.1)

# Le haut étant dégagé, le titre se place sans conflit
title("Quels paramètres influencent le plus la croissance ?", line = 1.8)
```

Quels paramètres influencent le plus la croissance ?



Règle de lecture d'une matrice démographique :

- **En colonne (X, temps t) :** le stade de **départ** de l'individu.
- **En ligne (Y, temps $t+1$) :** le stade d'**arrivée** l'année suivante.

Chaque cellule se lit donc : « De la colonne vers la ligne ».

- La **première ligne** correspond aux naissances (vers le stade *yearling*). Seules les femelles matures s'y projettent (fécondité de 0.04).
- La **diagonale** correspond aux individus qui survivent et restent dans le même stade d'une année sur l'autre (ex. adulte restant adulte : 0.95).
- La **sous-diagonale** correspond aux transitions de croissance (ex. juvénile devenant mature : 0.07).

Guide d'interprétation du graphique :

- **Comment lire la grille :**
  1. Choisissez un stade de départ sur l'axe du bas (t).
  2. Remontez verticalement vers le stade d'arrivée sur l'axe de gauche ($t+1$).

— **Distinction essentielle : Réalité biologique vs Sensibilité mathématique :**
  — **Transition réelle [juvenile → mature = 0.567] :** Forte sensibilité. Elle représente la maturation (le passage des grands jeunes vers le stock reproducteur).
  — **Transition réelle [mature → mature = 0.579] :** Très forte sensibilité. C'est le maintien des adultes reproducteurs déjà en place, pilier de la population.
  — **Cas mathématiques théoriques [post-reprod → mature = 0.581] ou [mature → juvenile = 0.387] :** La sensibilité brute calcule la dérivée mathématique même pour des cases nulles dans la nature. Une orque ménopausée ne redevient jamais mature, et un adulte ne rajeunit jamais en juvénile ($A_{ij} = 0$). En biologie de la conservation, **on ne prend en compte que les cases correspondant à des transitions réelles**.

1.5 À vous de jouer ! (Exercice autonome)

Vous maîtrisez désormais les outils de base. Vous allez les appliquer sur une espèce terrestre très étudiée en biologie de la conservation : **la Tortue du désert** (*Gopherus agassizii*).



FIGURE 1.3 : Tortue du désert (*Gopherus agassizii*), une espèce menacée dont la viabilité dépend fortement des taux de survie adulte (Source : Wikimedia Commons, CC-BY).

Contexte historique et gestionnaire (Doak et al., 1994) : Dans les années 1990, l'armée américaine souhaitait agrandir la base militaire de Fort Irwin, menaçant de détruire jusqu'à 13% de la population restante de Tortues du désert en Californie. Pour évaluer la viabilité de l'espèce, les écologues Doak, Kareiva et Klepetka (1994) ont construit des matrices structurées en 8 stades de taille.

Sur le terrain, la survie des adultes était connue, mais **le recrutement des jeunes tortues était quasi impossible à mesurer** car les nouveau-nés sont minuscules et passent inaperçus sous les buissons. Les auteurs ont donc testé 4 scénarios de fécondité (production annuelle de jeunes femelles par adulte) :

- `low` **:** Comptage brut des jeunes observés (très fortement sous-estimé par la détection imparfaite).
- `med.low` **:** Estimation corrigée (multipliée arbitrairement par 10 pour compenser le biais d'observation).
- `med.high` **et** `high` **:** Estimations basées sur une population témoin particulièrement saine et féconde (site de Goffs), représentatives d'années humides exceptionnelles.

Ces matrices sont stockées dans la liste `tortoise` du package `popbio`.

Votre mission : Exécutez le code ci-dessous étape par étape dans RStudio et répondez aux questions sur votre brouillon.

1. Découverte des données

```
data(tortoise)
# Affichons seulement la matrice du scénario "moyenne-haute" (med.high)
tortoise[["med.high"]]
```

```
          yearling  juv1  juv2  imm1  imm2 subadult adult1 adult2
yearling     0.000 0.000 0.000 0.000 0.000    1.300  1.980   2.57
juv1         0.716 0.567 0.000 0.000 0.000    0.000  0.000   0.00
juv2         0.000 0.149 0.567 0.000 0.000    0.000  0.000   0.00
imm1         0.000 0.000 0.149 0.604 0.000    0.000  0.000   0.00
imm2         0.000 0.000 0.000 0.235 0.560    0.000  0.000   0.00
subadult     0.000 0.000 0.000 0.000 0.225    0.678  0.000   0.00
adult1       0.000 0.000 0.000 0.000 0.000    0.249  0.851   0.00
adult2       0.000 0.000 0.000 0.000 0.000    0.000  0.016   0.86
```

Lecture biologique 8 stades de développement : de l'éclosion (yearling) aux très grands individus (adult2). La reproduction ne débute qu'au stade subadulte (subadult, carapace > 180 mm), ce qui prend entre 15 et 20 ans dans le désert !

2. Viabilité de la population Calculez le taux de croissance (λ) du scénario `med.high`. *Question : Cette population est-elle en croissance ou en déclin dans le cadre de ce scénario ?*

```
lambda(tortoise[["med.high"]])
```

3. Comparaison des scénarios La fonction `sapply()` permet d'appliquer la fonction `lambda` aux 4 matrices d'un seul coup. C'est magique !

```
sapply(tortoise, lambda)
```

Question : Que remarquez-vous sur la valeur de λ, même sous le scénario le plus optimiste (high) ? Si une réserve finance des couveuses artificielles ou des programmes de reproduction en captivité, cela suffira-t-il à sauver l'espèce ?

4. Quelle action de conservation privilégier ? Générez le graphique de sensibilité pour la population avec une "haute" fécondité.

```
# Astuce : remplacez "???" par le nom de la matrice "haute" ("high")
sens_tortue <- sensitivity(tortoise[["???"]])

image2(sens_tortue,
       log = FALSE,
       col = c("white", rev(heat.colors(23))),
       labels = c(1, 2),
       mar = c(4.5, 5, 3.5, 1),
       box.offset = 0.1)

title("Sensibilité - Tortue du désert", line = 1.8)
```

Question : Comparez la sensibilité de la reproduction (ligne 1) à celle de la survie des adultes (diagonale adult1). À l'époque, les gestionnaires voulaient abattre les corbeaux qui mangeaient les jeunes tortues. À la lumière de cette matrice, était-ce la stratégie la plus efficace ?

5. Boucler la boucle : Du terrain au modèle de projection

Vous êtes chargé de diagnostiquer la viabilité de la population de Cincles plongeurs que vous avez analysée au début du TP :

- Les suivis de nids indiquent un recrutement de **1.3 jeune femelle par adulte** et une survie juvénile en première année de **0.35**.
- Vous disposez de la survie adulte réelle estimée par votre modèle CMR ($\Phi = 0.56$).

Construisez la matrice de Leslie correspondante (2×2 : Juvéniles, Adultes) :

```
# 1. Matrice avec la survie estimée par le modèle CJS (Phi = 0.56)
mat_cincle <- matrix(c(???,    ???,
                       ???,  ???),
                     nrow = 2, byrow = TRUE)
colnames(mat_cincle) <- rownames(mat_cincle) <- c("Juvénile", "Adulte")

# Taux de croissance asymptotique
lambda(mat_cincle)
```

Question 1 : La population de cincle est-elle viable avec cette survie estimée (>1) ?

Question 2 : Si un observateur débutant n'avait pas corrigé la détection imparfaite (=0.90) et avait estimé la survie brute apparente à 0.50 au lieu de 0.56, quelle aurait été sa conclusion sur l'avenir de la population ?

i Défi : Modéliser le décrochage des trajectoires temporelles [Optionnel - Niveau Avancé]

Un écart de λ abstrait (1.010 vs 0.969) ne parle pas toujours aux décideurs. Pour convaincre un gestionnaire, rien ne vaut une courbe de projection comparée.
Votre défi :
À l'aide des fonctions vues plus haut pour l'Orque (`pop.projection`), construisez un script simulant l'évolution des deux populations sur 30 ans à partir d'un stock de départ identique de 100 oiseaux (50 juvéniles et 50 adultes).

Cahier des charges :

1. Définissez la durée (`duree <- 30`) et le vecteur initial `n_depart`.
2. Projetez les deux matrices (`mat_cincle_reelle` et `mat_cincle_naif`) sur cette période. *(Indice : pour couvrir les pas de temps de 0 à 30, demandez `iterations = duree + 1`).*
3. Extrayez l'abondance totale à chaque pas de temps via `colSums(proj$stage.vectors)` et assemblez-les dans un tableau (`tibble`) contenant une colonne `Annee` (de 0 à 30), une colonne `Reel` et une colonne `Naif`.
4. Passez ce tableau au format long avec `pivot_longer()` pour tracer les deux trajectoires sur un même graphique `ggplot2`.

(Besoin d'un coup de pouce ? Complétez le canevas ci-dessous en remplaçant les `???`) :

```r
# 1. Paramètres initiaux
n_depart <- c(50, 50)
duree    <- 30

# 2. Projections matricielles
proj_reel <- pop.projection(A = mat_cincle_reelle, n = n_depart, iterations = ???)
proj_naif <- pop.projection(A = mat_cincle_naif,   n = n_depart, iterations = ???)

# 3. Structuration des résultats
df_comparaison <- tibble(
  Annee = 0:duree,
  Reel  = colSums(proj_reel$???),
  Naif  = colSums(proj_naif$???)
) |>
  pivot_longer(cols = c(Reel, Naif), names_to = "Scenario", values_to = "Effectif_Total

# 4. Tracé ggplot2
ggplot(df_comparaison, aes(x = Annee, y = Effectif_Total, color = Scenario, linetype =
  geom_line(linewidth = 1.2) +
  geom_point(size = 2) +
  geom_hline(yintercept = sum(n_depart), linetype = "dotted", color = "grey40") +
  scale_color_manual(
    values = c("Reel" = "forestgreen", "Naif" = "firebrick"),
    labels = c("Comptage brut naïf (Phi = 0.50, faux déclin)", "Modèle CJS corrigé (Phi
  ) +
  scale_linetype_manual(
    values = c("Reel" = "solid", "Naif" = "dashed"),
    labels = c("Comptage brut naïf (Phi = 0.50, faux déclin)", "Modèle CJS corrigé (Phi
  ) +
  scale_x_continuous(breaks = seq(0, duree, by = 5)) +
  labs(
    title = "Impact de la correction CMR sur la trajectoire projetée du Cincle",
    subtitle = "Divergence exponentielle entre une population viable et un faux déclin"
    x = "Années de projection",
    y = "Effectif total d'oiseaux",
    color = "Approche",
    linetype = "Approche"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")
```

2 Analyse de cohortes

2.1 Objectifs

Cette section doit permettre d'illustrer la partie du cours de Population Dynamics de l'EC "Fonctionnement des écosystèmes" consacrée à l'analyse des cohortes. Vous utiliserez les tailles corporelles d'individus échantillonnés sur le terrain à plusieurs dates pour :

1. Produire la structure démographique instantanée de la population pour chaque date d'échantillonnage
2. Réaliser la décomposition polymodale pour identifier les cohortes à chaque date d'échantillonnage
3. Créer une courbe de croissance et une courbe de mortalité
4. Utiliser une relation allométrique pour passer de la taille des individus à leur masse
5. Produire la courbe d'Allen

Vous devrez en premier lieu importer les données fournies dans un fichier Excel et vous assurer qu'elles sont dans un format permettant les analyses et représentations graphiques.

2.2 Présentation de l'étude

Le suivi démographique des populations animales est extrêmement fréquent dans le domaine de l'écologie. Le suivi des populations naturelles est particulièrement pertinent dans un contexte de conservation, pour réaliser des études d'impact ou pour mesurer l'évolution de la biodiversité.

Ici, une population de gastéropodes *Nassarius reticulatus* (la nasse réticulée) a été suivie pendant 5 ans. Deux sessions d'échantillonnage ont été organisées chaque année depuis 2010 : la première a été réalisée en mars, juste après le recrutement

des juvéniles, et la seconde a été réalisée 6 mois plus tard, en septembre. Tous les échantillons ont été collectés au même endroit, selon la même méthode (collecte systématique de tous les individus présents à l'intérieur de 10 quadrats positionnés aléatoirement dans la zone d'étude), lors d'une marée basse de fort coefficient (marées de printemps et d'automne). Les individus échantillonnés ont été mesurés sur place, de la base, à l'apex de la coquille (voir Figure 2.1) à l'aide d'un pied à coulisse (précision : centième de millimètre) et relâchés sur place.

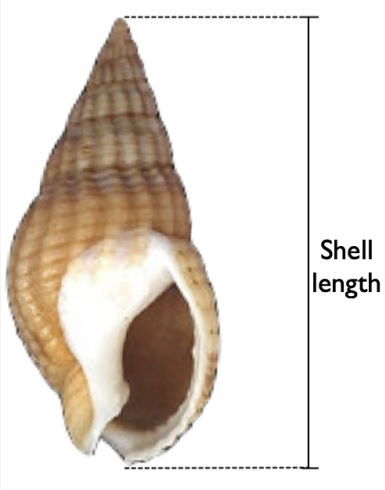


FIGURE 2.1 : Coquille de nasse réticulée *Nassarius reticulatus* (Linnaeus, 1758)

L'espèce étudiée présente les caractéristiques suivantes :

— Les individus ont une durée de vie de 5 ans en moyenne.
— Les plus grands individus peuvent atteindre une taille de plus de 40 millimètres.
— Il n'y a qu'une seule période de reproduction chaque année. Il n'y a donc qu'un unique recrutement chaque année, au tout début du mois de mars.

Des travaux antérieurs, réalisés au laboratoire, ont montré que la taille et la masse des individus étaient liées par la relation allométrique suivante :

$$w = 0,0013 \cdot l^{2,3}$$

avec w, la masse en grammes et l la longueur des coquilles en millimètres.

! Objectif principal

L'objectif principal de cette étude est de produire la courbe d'Allen d'une cohorte de cette population. Cette courbe sera utilisée pour déterminer des gains et des pertes de biomasses au sein de l'écosystème étudié. Les étapes nécessaires à la production de cette courbe sont présentées ci-dessous.

2.3 Avant de vous lancer...

Pour travailler dans de bonnes conditions, vous aurez absolument besoin de travailler dans un script et à l'intérieur d'un `Rproject`. Si vous ne savez plus comment faire, reportez-vous

aux chapitres correspondants du livre en ligne de Biométrie du semestre 3 : au sujet des scripts et au sujet des `Rprojects`. Vous devrez notamment (liste non exhaustive !) :

1. Créer un nouveau script (nommez-le `Nasses.R`)
2. Télécharger (si besoin) et charger quelques packages (voir plus bas)
3. Télécharger dans votre répertoire de travail le fichier Nassarius.csv
4. Importer dans `RStudio` les données du fichier `Nassarius.csv`

Pour cette section, vous aurez besoin des packages suivants :

— `tidyverse` : pour manipuler les données et faire des graphiques (Wickham et al. 2019)
— `mixdist` : pour effectuer les décompositions polymodales (Macdonald et Juan Du 2018)

Si vous ne savez plus comment installer et charger des packages en mémoire, reportez-vous au chapitre correspondant du livre en ligne de Biométrie du semestre 3

```
library(tidyverse)
library(mixdist)
```

Pour importer les données, utilisez l'assistant d'importation de `RStudio`. Notez que :

— Les colonnes sont séparées par des tabulations
— Le symbole utilisé pour les décimales dans le fichier `Nassarius.csv` est la virgule

Vous devrez donc spécifier correctement ces éléments pour pouvoir importer le fichier dans le logiciel. Si vous ne savez plus comment faire, reportez-vous au chapitre correspondant du livre en ligne de Biométrie du semestre 3.

Si l'importation s'est déroulée normalement, vous devriez maintenant disposer de l'objet nommé `Nassarius` suivant :

```
Nassarius
```

```
# A tibble: 3,710 x 2
    size date
```

```
   <dbl> <chr>
 1  4.8  2010-03
 2  3    2010-03
 3  5.56 2010-03
 4  3.74 2010-03
 5  4.1  2010-03
 6  2.21 2010-03
 7  2.75 2010-03
 8  3.18 2010-03
 9  2.56 2010-03
10  4.36 2010-03
# i 3,700 more rows
```

Et la commande suivante devrait produire exactement ces résultats :

```
Nassarius |>
  count(date)
```

```
# A tibble: 10 x 2
   date        n
   <chr>   <int>
 1 2010-03   468
 2 2010-09   268
 3 2011-03   481
 4 2011-09   276
 5 2012-03   460
 6 2012-09   265
 7 2013-03   468
 8 2013-09   258
 9 2014-03   487
10 2014-09   279
```

2.4 Les étapes de l'analyse

Pour produire une courbe d'Allen, de nombreuses étapes sont nécessaires. Vous devrez :

1. Produire la *structure démographique instantanée* de la population à chaque date d'échantillonnage

`|>` : le pipe est un opérateur spécial qui prend l'objet situé à gauche et le transmet à la fonction placée à droite, en guise de premier argument. P. ex. : `Nassarius |> count(date)` est équivalent à `count(Nassarius, date)`
`count()` : compte le nombre d'occurrences de chaque valeur possible (ou niveau/modalité) d'une variable catégorielle (ou facteur).

2. Réaliser la *décomposition polymodale* pour identifier les cohortes présentes dans la population à chaque date d'échantillonnage.
3. Déterminer la taille moyenne de la coquille des individus de la cohorte d'intérêt, à chaque date d'échantillonnage, pour produire la *courbe de croissance*
4. Déterminer l'abondance des individus de la cohorte d'intérêt, à chaque date d'échantillonnage, pour produire la *courbe de survie*

Je vais présenter ci-dessous les étapes de cette procédure **pour une unique date d'échantillonnage** : march 2010. Vous devrez reproduire ces étapes pour les 9 autres dates d'échantillonnage.

2.5 Sélection des données d'une date spécifique

Notre jeu de données `Nassarius` contient 2 colonnes : la première contient les dates d'échantillonnage et la seconde les tailles individuelles en millimètres. La première étape consiste à créer un nouvel objet (nous le nommerons `Nas_01`) qui contiendra uniquement les données collectées lors de la toute première session d'échantillonnage de mars 2010. Vous devriez déjà savoir comment utiliser la fonction `filter()` pour le faire :

```
Nas_01 <- Nassarius |>
  filter(date == "2010-03")

Nas_01
```

```
# A tibble: 468 x 2
    size date
   <dbl> <chr>
 1  4.8  2010-03
 2  3    2010-03
 3  5.56 2010-03
 4  3.74 2010-03
 5  4.1  2010-03
 6  2.21 2010-03
```

```
 7  2.75 2010-03
 8  3.18 2010-03
 9  2.56 2010-03
10  4.36 2010-03
# i 458 more rows
```

```
dim(Nassarius)
```

```
[1] 3710    2
```

`filter()` : permet de filtrer les lignes d'un tableau (ici `Nassarius`) pour ne conserver que celles qui remplissent une condition spécifiée par l'utilisateur (ici `date == "march_10"`)

```
dim(Nas_01)
```

```
[1] 468   2
```

Comme nous pouvons le constater, on passe du tableau original `Nassarius` contenant 3710 lignes à un nouveau tableau `Nas_01` qui en contient seulement 468.

`dim()` : Affiche le nombre de lignes et de colonnes d'un tableau.

2.6 Structure démographique instantanée

Maintenant que nous disposons d'une table `Nas_01` qui contient uniquement la taille des individus collectés en mars 2010, il nous faut visualiser la structure démographique instantanée afin de déterminer combien de cohortes étaient présentes dans la population à cette date. La structure démographique instantanée est ici simplement un histogramme présentant la distribution des tailles individuelles. Nous allons donc placer la taille des individus sur l'axe des `x` en guise de descripteur individuel, et sur l'axe des `y`, `RStudio` placera automatiquement l'abondance pour chaque classe de taille en guise de descripteur populationnel.

```
Nas_01 |>
  ggplot(aes(x = size)) +
  geom_histogram()
```

```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

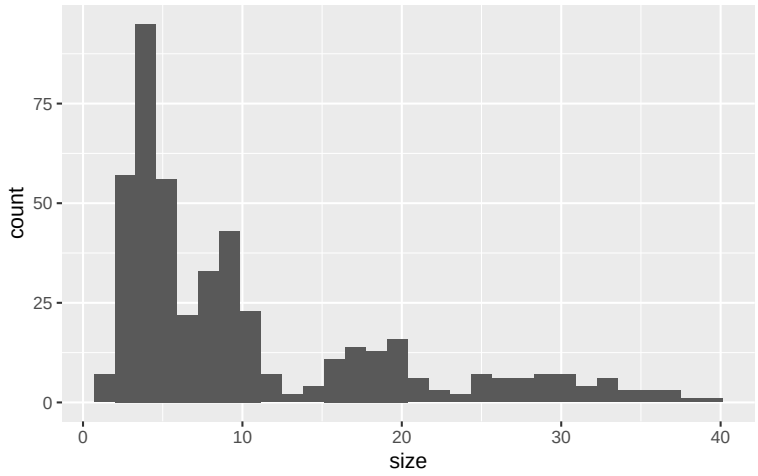


FIGURE 2.2 : Structure démographique instantanée

Notez le message d'avertissement qui s'affiche quand vous produisez cet histogramme. Il indique que `R` a choisi pour nous le nombre de classes de tailles. Par défaut, il crée 30 classes, mais nous indique que ce n'est certainement pas le meilleur choix. Nous allons donc devoir créer nous-mêmes manuellement les classes de tailles pour (i) identifier les cohortes présentes dans la population et (ii) fixer des classes identiques que nous utiliserons pour toutes les autres dates d'échantillonnage et qui rendront les comparaisons plus aisées.

`ggplot()` : crée un graphique
`aes()` : associe une variable d'un jeu de données à une caractéristique esthétique d'un graphique (p. ex. la position le long de l'axe des `x` ou des `y`, la couleur, la forme, la taille, etc.)
`geom_histogram()` : ajoute un objet géométrique de type histogramme à un graphique produit par `ggplot()`

Puisque les individus de cette espèce peuvent atteindre une taille de 40 millimètres environ, nous allons définir des classes de tailles tous les millimètres. On peut faire ça simplement en créant un vecteur qui contient les limites des classes de tailles que l'on souhaite :

```
# Calcul de la taille maximale observée
taille_max <- max(Nassarius$size, na.rm = TRUE)
taille_max
```

```
[1] 41.33
```

`max()` : affiche la valeur maximale contenue dans un vecteur

```
# Définition des limites des classes de tailles
limites <- 0:(taille_max + 1)
limites
```

```
 [1]  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
[26] 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42
```

L'objet `limites` contient les limites des classes de tailles que nous utiliserons pour produire les structures démographiques instantanées, pour chaque date d'échantillonnage, et pas seulement pour mars 2010. C'est la raison pour laquelle nous avons utilisé `max(Nassarius$size) +1` : cette syntaxe nous assure que nos classes de tailles recouvrent bien la taille de tous les individus échantillonnés au cours des 5 années d'études. Ainsi, nous pourrons utiliser les mêmes classes de tailles pour toutes les dates.

: : l'opérateur "deux points" permet de créer des suites d'entiers

Maintenant, on peut utiliser le vecteur `limites` comme argument de la fonction `geom_histogram()`. On modifie aussi les couleurs des barres de l'histogramme pour mieux visualiser les classes :

```
Nas_01 |>
  ggplot(aes(x = size)) +
  geom_histogram(breaks = limites, color = "black", fill = "steelblue")
```

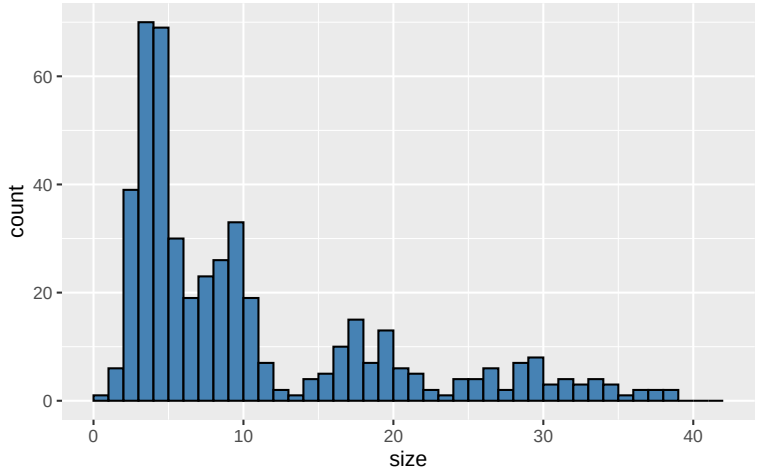


FIGURE 2.3 : Structure démographique instantanée utilisable pour la décomposition polymodale

Cette structure démographique instantanée (Figure 2.3) fait apparaître 5 cohortes, bien que la plus âgée soit à peine visible. Les pics, ou modes, sont situés autour de 4, 9, 17, 30 et 34 millimètres. Ces valeurs approximatives sont importantes car nous les utiliserons pour réaliser la décomposition polymodale.

2.7 Décomposition polymodale

2.7.1 Principe et étapes

Effectuer une décomposition polymodale revient à tenter d'ajuster une distribution Normale à chaque cohorte de la population. Puisque nous avons identifié 5 cohortes dans la population en mars 2010, il nous faut identifier 5 distributions Normales distinctes dont la somme reproduira le plus fidèlement possible la structure démographique instantanée observée. Plusieurs étapes sont requises pour y parvenir.

Nous allons utiliser le package `mixdist`, que vous devez avoir déjà installé et chargé en mémoire. Ce package fournit plusieurs fonctions relativement simples d'utilisation et qui permettent d'ajuster des distributions Normales à une distribution observée (notre structure démographique instantanée).

La fonction du package `mixdist` qui nous permettra de réaliser la décomposition polymodale est la fonction `mix()`. Son fichier d'aide nous indique que pour fonctionner correctement, il faut lui fournir plusieurs choses :

```
?mix()
```

`?` : l'opérateur "point d'interrogation" suivi du nom d'une fonction, permet d'ouvrir le fichier d'aide de cette fonction.

Les deux premiers arguments de la fonction `mix()` ne possèdent pas de valeur par défaut. Nous devons donc spécifier nous-mêmes ces deux arguments :

1. Le premier s'appelle `mixdat`. Il doit obligatoirement s'agir d'un tableau de données (`data.frame` ou `tibble`) contenant 2 colonnes. La première colonne doit contenir les limites supérieures des classes de tailles de notre structure démographique instantanée. Le dernier élément de cette colonne doit être "ouvert" : il doit être fixé à $+\infty$. La seconde colonne doit contenir les abondances pour chaque classe de taille de la structure démographique instantanée. Nous allons voir juste après comment créer cet objet `mixdat`.
2. Le second argument s'appelle `mixpar`. Là encore, il s'agit d'un tableau de données. Il doit contenir des valeurs approchées pour les paramètres des distributions Normales que nous souhaitons ajuster à chacune

de nos 5 cohortes. Chaque distribution Normale possède 2 paramètres : la moyenne (qui correspond à la position du pic sur l'axe des `x` d'un histogramme) et l'écart-type (qui correspond à l'étalement de la courbe en cloche, c'est-à-dire à la dispersion des données de part et d'autre de la moyenne). Là encore, nous verrons plus bas comment créer cet objet.

2.7.2 Création du premier objet

Pour créer le premier tableau qui sera utilisé comme argument `mixdat` de la fonction `mix()`, nous allons nous servir de la fonction `mixgroup()` du package `mixdist`. Elle est très simple à utiliser puisqu'elle n'a besoin que de 2 arguments :

1. Un vecteur de données (ici, la taille de tous les individus échantillonnés en mars 2010)
2. La liste des valeurs correspondant aux limites des classes de tailles de la structure démographique instantanée. Nous disposons déjà de cet objet puisque nous l'avons créé plus tôt : c'est le vecteur `limites`

```
mix_01 <- mixgroup(Nas_01$size, breaks = limites)
class(mix_01)
```

```
[1] "mixdata"    "data.frame"
```

L'objet `mix_01` que nous venons de créer est donc un `data.frame`, mais c'est aussi un objet de classe `mixdata` que nous utiliserons en guise de premier argument de la fonction `mix()`. Pour afficher son contenu, il suffit comme toujours de taper son nom :

```
mix_01
```

```
     X count
1    1     1
2    2     6
3    3    39
4    4    70
5    5    69
```

```
6     6    30
7     7    19
8     8    23
9     9    26
10   10    33
11   11    19
12   12     7
13   13     2
14   14     1
15   15     4
16   16     5
17   17    10
18   18    15
19   19     7
20   20    13
21   21     6
22   22     5
23   23     2
24   24     1
25   25     4
26   26     4
27   27     6
28   28     2
29   29     7
30   30     8
31   31     3
32   32     4
33   33     3
34   34     4
35   35     3
36   36     1
37   37     2
38   38     2
39   39     2
40   40     0
41   41     0
42  Inf     0
```

Notez que la dernière ligne de `mix_01` correspond à la catégorie [42 mm ; $+\infty$[. Pour vérifier que nous n'avons pas fait d'erreur, on peut faire une représentation graphique de cet objet particulier avec la fonction `plot()` :

```r
plot(mix_01, xlab = "Size (mm)", ylab = "Probability")
```

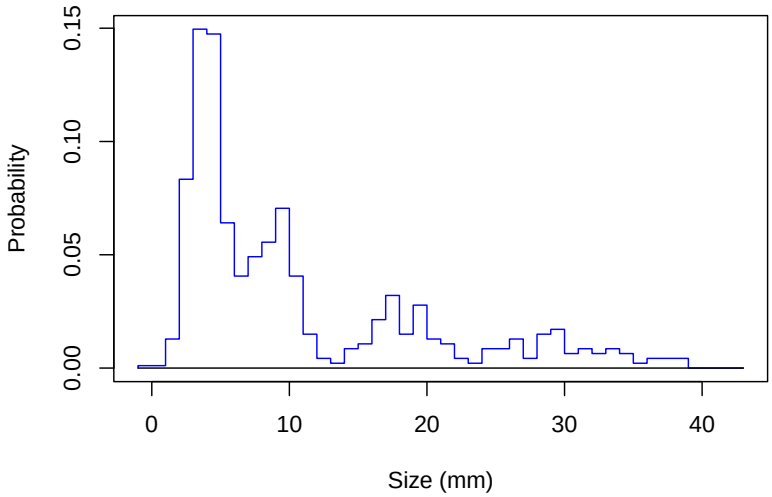


FIGURE 2.4 : Le contenu de `mix_01` correspond exactement à la structure démographique instantanée de mars 2010. La ligne bleue est le contour de la structure démographique instantanée, qui devra être approchée par la superposition de 5 courbes Normales correspondant aux 5 cohortes qui composent la population en mars 2010

Au final, l'objet `mix_01` est simplement la structure démographique instantanée de mars 2010, présentée dans un format qui sera compris par la fonction `mix()` que nous utiliserons pour effectuer la décomposition polymodale.

`plot()` : fonction générique permettant de produire des graphiques sans `ggplot2`. La forme du résultat dépendra de la classe de l'objet utilisé comme argument.

2.7.3 Création du deuxième objet

Pour créer le second `data.frame()` dont la fonction `mix()` aura besoin, nous allons utiliser une autre fonction du package `mixdist` : la fonction `mixparam()`. Cette fonction prend au minimum 2 arguments :

1. `mu` : un vecteur qui contient la position approximative des modes (ou pics) de chaque cohorte. Si nous avons 5 cohortes à identifier, il faudra fournir 5 valeurs pour `mu`. Pour mars 2010, nous utiliserons les valeurs présentées plus haut, au niveau de la Figure 2.3 (4, 9, 17, 30 et 34 millimètres). Ces valeurs changeront pour chaque date d'échantillonnage et il faudra donc examiner attentivement les structures démographiques instantanées de chaque échantillonnage pour les déterminer. J'insiste sur le fait que la position des pics peut être approximative, mais qu'elle ne doit malgré tout pas être trop éloignée des vraies valeurs.
2. `sigma` : un vecteur qui contient la valeur approchée de l'écart-type de chaque cohorte. L'écart-type d'une

cohorte correspond à l'étalement des tailles de part et d'autre de la moyenne de la cohorte. C'est ce que nous avons appelé "polymorphisme" dans le cours de dynamique des populations, puisque cet étalement représente des performances de croissance variables pour des individus qui sont tous nés approximativement en même temps. Là encore, il faut fournir autant de valeurs que nous avons de cohortes (soit 5 pour mars 2010). Puisque nous n'avons pas d'idée précise d'ordre de grandeur pour ces écarts-types, nous utiliserons la valeur `1` pour chaque cohorte.

Ainsi, on obtient le second `data.frame` ainsi :

```
param_01 <- mixparam(mu = c(4, 9, 17, 30, 34),
                     sigma = c(1, 1, 1, 1, 1))
param_01
```

```
  pi mu sigma
1 0.2  4     1
2 0.2  9     1
3 0.2 17     1
4 0.2 30     1
5 0.2 34     1
```

La première colonne de ce nouveau `data.frame`, nommée `pi`, correspond à la proportion de l'effectif total échantillonné, contenu dans chaque cohorte. Puisque nous n'avons rien spécifié, `mixparam()` suppose que chaque cohorte contient une proportion identique des individus de la population, et fixe donc la proportion à 0.2 (soit 20% de l'abondance totale dans chacune des 5 cohortes supposées). Nous savons pertinemment que ces proportions sont fausses puisqu'en réalité, l'abondance au sein des cohortes décroit avec l'âge des individus en raison de la mortalité. On sait donc que la cohorte la plus jeune sera la plus abondante, et que les cohortes plus âgées seront de moins en moins abondantes. Ces proportions seront estimées automatiquement par la fonction `mix()` lorsque nous réaliserons la décomposition polymodale. Les autres colonnes de `param_01`, qui contiennent les valeurs que nous avons fournies manuellement, seront également ajustées lors de la décomposition polymodale

`mixparam()` : fonction qui crée un `data.frame` dans lequel chaque ligne contient les caractéristiques approchées d'une cohorte de la population échantillonnée
`c()` : fonction qui permet de créer un vecteur, donc une collection d'éléments qui sont tous du même type (ici, des valeurs numériques).

2.7.4 Ajustement des lois Normales aux données

Maintenant que nous disposons des deux objets nécessaires, nous pouvons réaliser la décomposition polymodale qui consiste, grâce à la fonction `mix()`, à ajuster une distribution Normale à chaque cohorte supposée de la population échantillonnée.

```
res_01 <- mix(mix_01, param_01)
```

```
Warning in mix(mix_01, param_01): The optimization process terminated because
iteration limit exceeded
```

```
res_01
```

```
Parameters:
       pi     mu sigma
1 0.46418  3.903 1.086
2 0.27062  8.795 1.555
3 0.14355 18.115 2.102
4 0.09100 28.231 2.861
5 0.03066 35.021 2.344

Distribution:
[1] "norm"

Constraints:
   conpi    conmu consigma
  "NONE"   "NONE"   "NONE"
```

La fonction `mix()` peut produire quelques avertissements. Ignorez-les à ce stade : seuls les messages d'erreurs sont problématiques et nous y reviendrons plus tard. L'objet `res_01` contient donc les résultats de la décomposition polymodale. C'est une liste de 3 éléments dont seul le premier nous intéresse.^[Il s'appelle `parameters`, sans majuscule, même si `R` affiche son nom avec une majuscule. Vous pouvez le vérifier en tapant `str(res_01)`.

`mix()` : réalise la décomposition polymodale. La fonction identifie une combinaison de distributions Normales qui s'ajuste le mieux possible aux données observées compte tenu des données brutes et des caractéristiques approximatives de chaque distribution Normale fournie par l'utilisateur.

`str()` : affiche la **str**ucture interne d'objets complexes.] Il contient les valeurs ajustées pour les 3 paramètres des distributions Normales (`pi`, `mu` et `sigma`) pour chacune des 5 cohortes identifiées en mars 2010.

Ainsi, par exemple, pour l'échantillon de mars 2010, la première ligne du tableau `parameters` contenu dans `res_01` nous apprend les choses suivantes :

```
res_01$parameters
```

```
          pi        mu    sigma
1 0.46417552  3.903201 1.085990
2 0.27061610  8.795297 1.555004
3 0.14354832 18.114948 2.101807
4 0.09099874 28.230516 2.861214
5 0.03066132 35.020985 2.343520
```

- La première cohorte représente 46.42% de l'abondance totale de la population
- La taille moyenne des individus composant cette cohorte vaut 3.9 millimètres
- L'écart-type (ou polymorphisme de taille) de cette cohorte vaut 1.1 millimètres

Les distributions Normales qui ont été ajustées peuvent être visualisées grâce à la fonction `plot()` :

```
plot(res_01)
```

On observe sur la Figure 2.5 et dans l'objet `res_01$parameters` que la qualité de l'ajustement (et donc de la décomposition polymodale) est bonne :

- la courbe verte représente bien les données observées : elle est bien ajustée aux contours de l'histogramme.
- l'abondance des cohortes décroit avec l'âge de la cohorte : `pi` décroit de la cohorte la plus jeune à la cohorte la plus âgée.
- la taille moyenne des individus augmente avec l'âge de la cohorte : `mu` augmente de la cohorte la plus jeune à la cohorte la plus âgée.
- le polymorphisme (ou dispersion des tailles autour de la moyenne) augmente avec l'âge de la cohorte : `sigma`

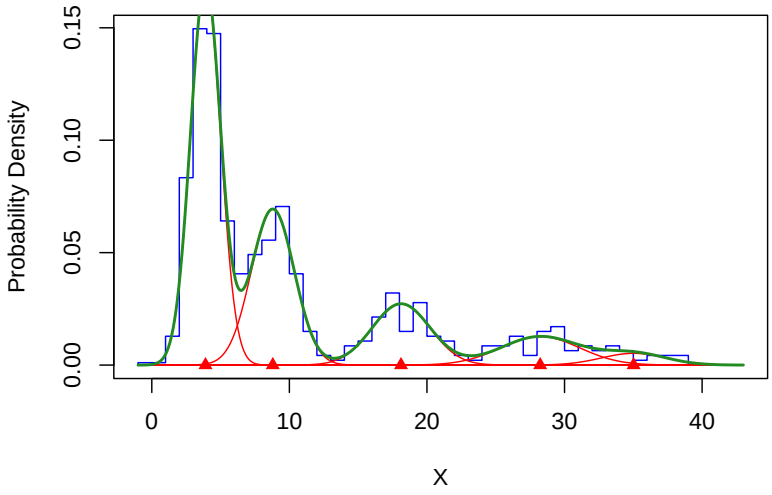


FIGURE 2.5 : Distribution observée (histogramme bleu) et cohortes ajustées (courbes rouges). Les triangles indiquent la position du mode des cohortes et la courbe verte est la somme de toutes les distributions Normales. Idéalement, cette courbe devrait être étroitement ajustée aux données observées

augmente de la cohorte la plus jeune à la cohorte la plus âgée.

! De l'importance du choix des valeurs initiales

Si nous avions choisi d'autres valeurs approchées lors de la création de l'objet `param_01` avec la fonction `mixparam()`, l'ajustement obtenu aurait pu être différent. La décomposition polymodale n'est pas une science exacte et il est souvent nécessaire de procéder par tâtonnements pour trouver des valeurs satisfaisantes. En particulier, les résultats que nous obtenons dépendent :

- du choix des valeurs pour la fonction `mixparam()`
- du choix des classes de tailles pour la structure démographique instantanée. Ici, nous avons des classes de tailles de un millimètre de large, mais nous aurions pu faire un autre choix (1,5 millimètre de large, ou 2) et nous aurions alors obtenu des résultats différents.

Il est donc important de retenir que les résultats obtenus ne sont pas "justes" ou "faux" en tant que tels et qu'il n'y a pas qu'une seule "bonne réponse". La qualité des résultats obtenus s'apprécie au regard de ce qu'on connait de l'espèce étudiée (traits d'histoire de vie, période de reproduction et de recrutement, nombre de cohortes supposées selon la date, etc.), et de ce qu'on connait du comportement "normal" des cohortes (baisse de l'abondance avec l'âge, augmentation de la taille moyenne avec

l'âge, augmentation du polymorphisme avec l'âge).

2.7.5 Et en cas de message d'erreur ?

Lorsque vous utiliserez la fonction `mix()`, il se peut que des messages d'erreurs et/ou des messages d'avertissement apparaissent.

ℹ Les messages d'information et les avertissements commencent en général par le mot `Warning` et sont presque toujours sans conséquence. La commande a été comprise par `RStudio` et un résultat a été produit. Dans le cas de la fonction `mix()`, les avertissements indiquent que la solution trouvée (l'ajustement des lois Normales aux cohortes observées) n'est peut-être pas optimale, mais une solution a néanmoins été obtenue.

💣 Les messages d'erreurs commencent par `Erreur` ou `Error` et indiquent que la commande n'a pas abouti. Dans le cas de la fonction `mix()`, cela peut être lié à 2 choses :

1. soit les valeurs choisies pour la fonction `mixparam()` sont trop éloignées de la position des pics réels, et la fonction `mix()` ne parvient donc pas à trouver de solution satisfaisante
2. soit le nombre de valeurs choisies pour la fonction `mixparam()` n'est pas le bon. Par exemple, s'il y a 5 cohortes et qu'on ne fournit que 4 valeurs pour `mu` et `sigma`, un message d'erreur apparaîtra. De même, s'il y a 4 cohortes et qu'on fournit 5 valeurs pour `mu` et `sigma`, un message d'erreur apparaîtra. Enfin, si on ne fournit pas le même nombre de valeurs pour `mu` et pour `sigma`, un message d'erreur apparaîtra.

Dans les deux cas, vous devez

1. revenir à votre structure démographique instantanée et l'observer plus attentivement
2. déterminer des valeurs plus appropriées[1] pour la fonction `mixparam()`
3. ré-exécuter le code depuis la création de l'objet `param_01` et jusqu'à la décomposition polymodale avec la fonction `mix()`

[1] ajoutez ou retirez une cohorte si besoin, et choisissez des valeurs plus proches des pics observés

2.8 Taille moyenne et abondance des cohortes

À partir des résultats de la décomposition polymodale, nous devons maintenant calculer l'abondance (*i.e.* le nombre d'individus) de chaque cohorte. Nous devrons ensuite stocker dans un nouveau tableau :

1. la date d'échantillonnage (mars 2010)
2. la valeur d'abondance de la cohorte dont on souhaite réaliser le suivi (il s'agit de la cohorte la plus jeune en mars 2010)
3. la taille moyenne des individus de la cohorte dont on souhaite réaliser le suivi[2]

[2] on sait déjà que cette taille vaut 3.9 millimètres en mars 2010.

L'abondance d'une cohorte est obtenue en multipliant la valeur de `pi` de la cohorte d'intérêt par le nombre total d'individus échantillonnés en mars 2010. En effet, sur la Figure 2.5, la surface totale comprise entre l'axe des abscisses et la courbe verte vaut 1. Cette surface correspond au nombre total d'individus échantillonnés en mars 2010 :

```
nrow(Nas_01)
```

```
[1] 468
```

Puisque la première cohorte représente 46.42% de l'abondance totale[3], tout ce dont on a besoin pour connaitre l'abondance de la cohorte la plus jeune est :

```
res_01$parameters[1, 1] * nrow(Nas_01)
```

```
[1] 217.2341
```

En arrondissant à l'entier le plus proche, on obtient :

```
round(res_01$parameters[1, 1] * nrow(Nas_01), 0)
```

```
[1] 217
```

[3] `nrow()` : affiche le nombre de lignes d'une matrice ou d'un `data.frame`. C'est ce que nous dit la valeur de `pi` de la première cohorte dans le tableau `res_01$parameters` :

pi	mu	sigma
0.4642	3.9032	1.0860
0.2706	8.7953	1.5550
0.1435	18.1149	2.1018
0.0910	28.2305	2.8612
0.0307	35.0210	2.3435

Nous savons donc maintenant que la première cohorte, la plus jeune de la population échantillonnée en mars 2010, a donc une taille moyenne de 3.9 millimètres et une abondance de 217 individus. Nous allons maintenant utiliser ces valeurs pour créer un tableau et placer le premier point des courbes de croissance, de survie et d'Allen.

`round()` : arrondit des valeurs numériques. Le deuxième argument permet d'indiquer le nombre de décimales.

2.9 Tableau et mise en forme des données

Nous venons de décrire ci-dessus la méthode que vous devrez appliquer pour chaque date d'échantillonnage. En suivant les mêmes étapes, vous devriez être en mesure d'obtenir la taille et l'abondance moyennes de notre cohorte d'intérêt pour les 9 dates restantes. Lorsque vous effectuerez la décomposition polymodale pour ces 9 dates, n'oubliez jamais que vous voulez suivre *systématiquement la même cohorte dans le temps*. Cette cohorte va progressivement évoluer vers des tailles plus importantes puisque les individus grandissent chaque mois et chaque année. Notre cohorte d'intérêt va donc progressivement se décaler vers la droite des structures démographiques instantanées, et les informations de cette cohorte ne seront donc pas systématiquement situées sur la première ligne des résultats de la décomposition polymodale.

Pour chaque date, on s'attend donc à ce que la taille moyenne des individus soit plus grande que la taille obtenue pour la date d'échantillonnage précédente. De même, l'abondance devrait diminuer au fil du temps en raison de la mortalité naturelle qui affecte tous les individus de la population.

Pour produire les 3 courbes (croissance, survie et Allen) dont nous avons besoin, nous allons créer un `tibble` contenant 3 colonnes :

1. la date d'échantillonnage
2. la taille moyenne des individus de la cohorte en millimètres
3. l'abondance de la cohorte (nombre d'individus de la cohorte)

Nous pouvons utiliser la fonction `tribble()` pour le faire :

`tribble()` : permet de créer un `tibble` ligne par ligne. Le "r" de `tribble` est l'abréviation de "**r**ow".

```
cohort <- tribble(
  ~date,        ~size,                   ~abundance,
  "2010-03-01", res_01$parameters[1,2], res_01$parameters[1,1] * nrow(Nas_01)
)
```

Pour chaque nouvelle date d'échantillonnage, vous devrez compléter ce tableau en ajoutant une nouvelle ligne à l'intérieur de cette fonction `tribble()`.

> ⚠ Attention !
>
> Une erreur fréquente est de saisir les données obtenues à chaque date d'échantillonnage dans un tableau différent à chaque fois. Ça n'est pas ce qu'il faut faire ! Il faut au contraire compléter le tableau `cohort` en ajoutant une nouvelle ligne au tableau existant de la façon suivante :
>
> ```
> cohort <- tribble(
>   ~date,        ~size,                   ~abundance,
>   "2010-03-01", res_01$parameters[1,2], res_01$parameters[1,1] * nrow(Nas_01),
>   "2010-09-01", ...                    , ...
> )
> ```

Nous devons spécifier une dernière chose afin de produire les graphiques : à ce stade, la colonne `date` du nouveau tableau `cohort` est considérée comme une variable de type `character` (type `<chr>`) :

```
cohort
```

```
# A tibble: 1 x 3
  date        size abundance
  <chr>      <dbl>     <dbl>
1 2010-03-01  3.90      217.
```

Il nous faut donc la transformer pour que `R` la reconnaisse comme étant une variable temporelle afin que les données apparaissent dans l'ordre chronologique (et non alphabétique)

sur l'axe des abscisses de nos graphiques. Voilà comment procéder :

```
# On charge le package lubridate pour travailler avec des dates
# Ce package fait partie du tidyverse
# Si vous l'avez installé, lubridate est disponible sur votre ordinateur
library(lubridate)
cohort <- cohort |>
  mutate(date = date(date))
```

On vérifie que la variable `date` possède maintenant le type `<date>` :

`mutate()` : crée de nouvelles variables dans un `tibble`, ou modifie des variables existantes.
`date()` : transforme des variables de type `<chr>` (caractères) en variables de type `<date>` (dates).

```
cohort
```

```
# A tibble: 1 x 3
  date        size abundance
  <date>     <dbl>     <dbl>
1 2010-03-01  3.90      217.
```

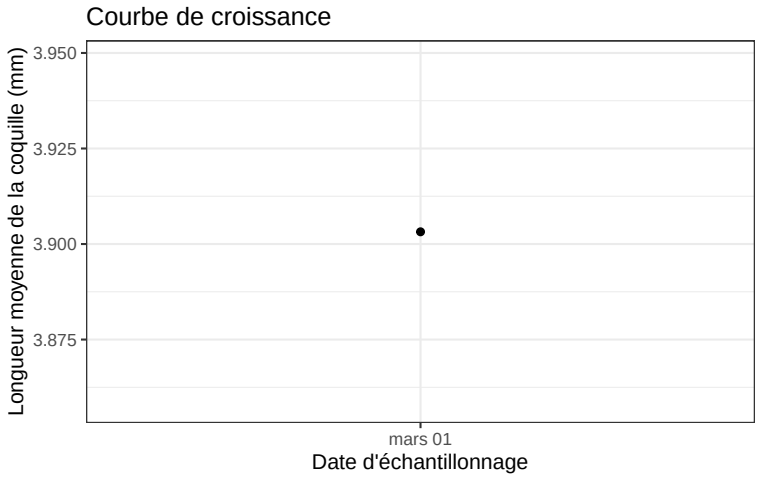
À ce stade, nous avons tout ce qu'il nous faut pour produire les courbes de croissance, de survie et d'Allen, à l'aide de `ggplot2`. Évidemment, chaque courbe ne contiendra ici qu'un seul point puisque nous avons examiné pour l'instant qu'une seule date d'échantillonnage. Elles ne seront complètes que lorsque vous aurez répété ce travail pour l'ensemble des 10 dates d'échantillonnage.

2.10 Courbe de croissance

On place les dates d'échantillonnage sur l'axe des abscisses, et la taille moyenne des individus de la cohorte d'intérêt sur l'axe des ordonnées :

```
cohort |>
  ggplot(aes(x = date, y = size, group = 1)) +
  geom_point() +
  geom_line() +
  labs(x = "Date d'échantillonnage",
       y = "Longueur moyenne de la coquille (mm)",
       title = "Courbe de croissance") +
```

```
  theme_bw()
```



`geom_point()` et `geom_line()` : permettent d'ajouter des points et des lignes (respectivement) sur un graphique

`labs()` : permet de spécifier le titre d'un graphique et des axes

`theme_bw()` : change l'apparence générale du graphique

L'argument `group = 1` est utilisé pour indiquer que toutes les dates d'échantillonnage appartiennent à la même série temporelle, et qu'on souhaite donc relier les dates par une ligne. Le résultat sera nettement plus parlant quand vous aurez ajouté des données d'autres dates d'échantillonnage sur le graphique, afin de visualiser l'évolution de la taille moyenne des individus de la cohorte d'intérêt au fil du temps.

2.11 Courbe de survie

Pour produire la courbe de survie, on procède de la même façon, mais on place les dates d'échantillonnage sur l'axe des abscisses, et l'abondance de la cohorte d'intérêt sur l'axe des ordonnées :

```
cohort |>
  ggplot(aes(x = date, y = abundance, group = 1)) +
  geom_point() +
  geom_line() +
  labs(x = "Date d'échantillonnage",
       y = "Abondance de la cohorte",
       title = "Courbe de survie") +
  theme_bw()
```

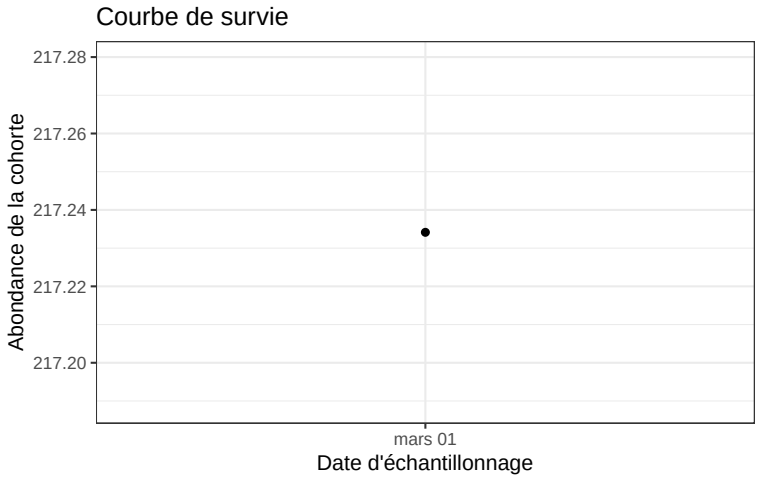


FIGURE 2.7 : Courbe de survie d'une cohorte de la population de *Nassarius reticulatus*, suivie pendant 5 ans. À compléter avec les données des 9 autres dates d'échantillonnage

2.12 Courbe d'Allen

Pour produire la courbe de survie, on place la taille moyenne des individus de la cohorte sur l'axe des abscisses, et l'abondance de la cohorte d'intérêt sur l'axe des ordonnées :

```
cohort |>
  ggplot(aes(x = size, y = abundance, group = 1)) +
  geom_point() +
  geom_line() +
  labs(x = "Longueur moyenne de la coquille (mm)",
       y = "Abondance de la cohorte",
       title = "Courbe d'Allen") +
  theme_bw()
```

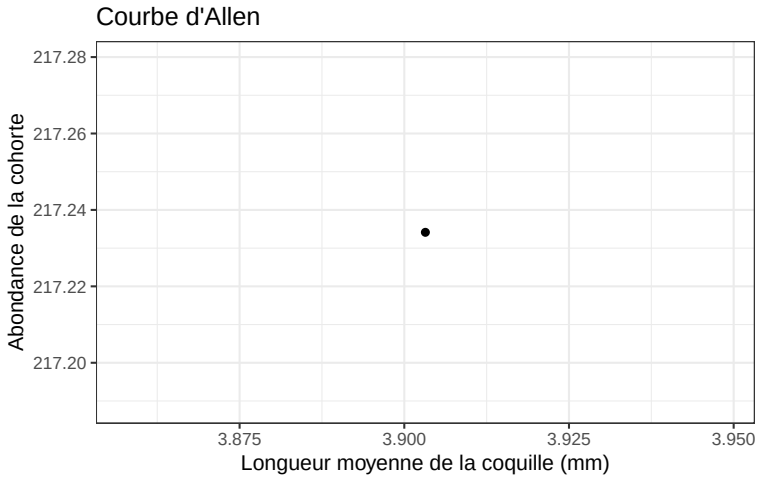


FIGURE 2.8 : Courbe d'Allen d'une cohorte de la population de *Nassarius reticulatus*, suivie pendant 5 ans. À compléter avec les données des 9 autres dates d'échantillonnage

2.13 Relation allométrique

L'un des objectifs de ce travail était de produire une courbe d'Allen pour étudier les **variations de biomasses**. Pour y parvenir, nous devons faire une courbe d'Allen sur laquelle figure la masse moyenne des individus de la cohorte d'intérêt (en grammes), plutôt que la taille moyenne des individus de cette cohorte (en millimètres). Pour passer des tailles en millimètres aux masses en grammes, il nous suffit d'appliquer la relation allométrique fournie dans la Section 2.2, et d'ajouter une nouvelle colonne à notre tableau grâce à la fonction `mutate()` :

```
cohort_2 <- cohort |>
  mutate(weight = 0.0013 * size ^ 2.3)

cohort_2
```

```
# A tibble: 1 x 4
  date        size abundance weight
  <date>     <dbl>     <dbl>  <dbl>
1 2010-03-01  3.90      217. 0.0298
```

Il n'y a plus qu'à utiliser cette nouvelle variable sur l'axe des abscisses d'une courbe d'Allen :

```
cohort_2 |>
  ggplot(aes(x = weight, y = abundance, group = 1)) +
  geom_point() +
  geom_line() +
  labs(x = "Masse moyenne des individus (g)",
       y = "Abondance de la cohorte",
       title = "Courbe d'Allen") +
  theme_bw()
```

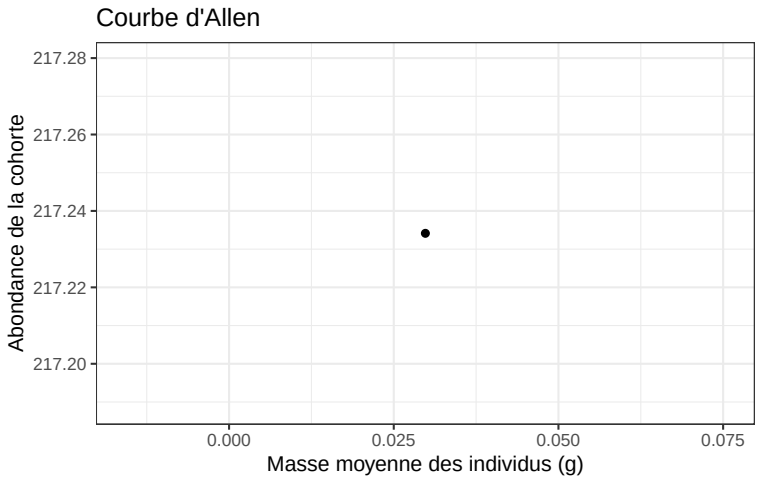
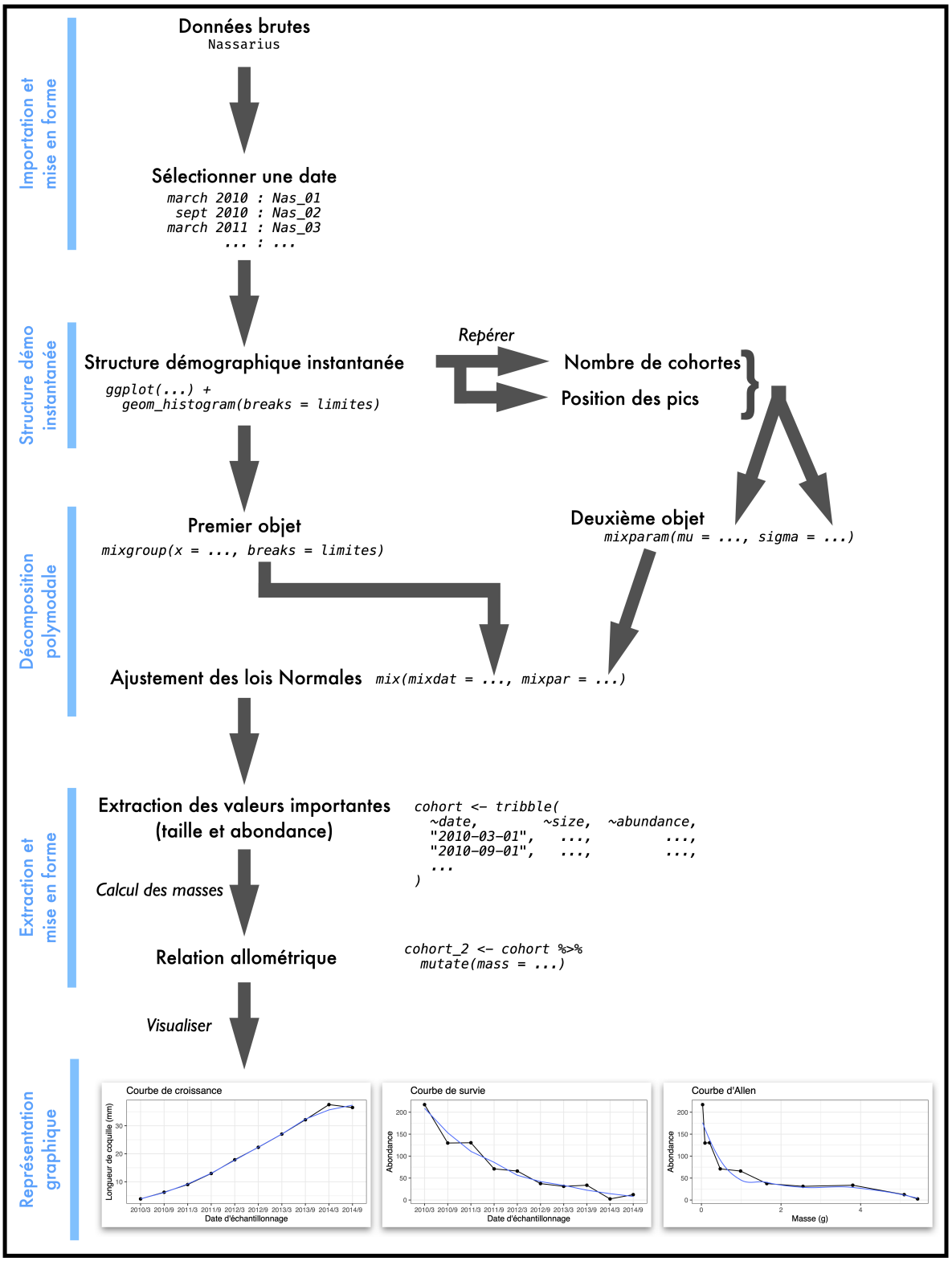


FIGURE 2.9 : Courbe d'Allen d'une cohorte de la population de *Nassarius reticulatus*, suivie pendant 5 ans. À compléter avec les données des 9 autres dates d'échantillonnage

2.14 À vous de jouer !

Au final, voilà comment on peut résumer les différentes étapes de ce travail.

Assurez-vous que vous avez bien compris chaque étape de la méthode décrite pour l'échantillonnage de mars 2010, et pourquoi on fait les choses présentées ici dans l'ordre où on les fait. À ce stade, vous devriez déjà avoir un script assez long qui devrait contenir la plupart des commandes évoquées plus haut. Vous devriez donc pouvoir reproduire ces étapes pour toutes les autres dates d'échantillonnage en faisant des copier-coller et en modifiant quelques éléments précis (dates, nom des objets, valeurs de tailles moyennes pour les cohortes, etc.). Attention aussi à adopter une structure de script la plus claire possible, afin de pouvoir corriger les éventuels bugs ou problèmes plus facilement.


Importation et mise en forme
Structure démo instantanée
Décomposition polymodale
Extraction et mise en forme
Représentation graphique
Données brutes
Nassarius
Sélectionner une date
march 2010 : Nas_01
sept 2010 : Nas_02
march 2011 : Nas_03
... : ...
Structure démographique instantanée
ggplot(...) +
geom_histogram(breaks = limites)
Repérer
Nombre de cohortes
Position des pics
Premier objet
mixgroup(x = ..., breaks = limites)
Deuxième objet
mixparam(mu = ..., sigma = ...)
Ajustement des lois Normales
mix(mixdat = ..., mixpar = ...)
Extraction des valeurs importantes
(taille et abondance)
cohort <- tribble(
~date, ~size, ~abundance,
"2010-03-01", ..., ...,
"2010-09-01", ..., ...,
...
)
Calcul des masses
Relation allométrique
cohort_2 <- cohort %>%
mutate(mass = ...)
Visualiser
Courbe de croissance
Longueur de coquille (mm)
Date d'échantillonnage
Courbe de survie
Abondance
Date d'échantillonnage
Courbe d'Allen
Abondance
Masse (g)

Il ne vous reste donc plus qu'à reproduire ce travail pour les 9 autres dates d'échantillonnage afin (i) de récupérer les valeurs d'abondance, de taille et de masse moyenne des individus de la cohorte d'intérêt (celle qui a été recrutée en mars 2010) et (ii) de compléter les 3 courbes que nous avons commencées plus haut.

Bon courage !

3 Systèmes dynamiques

3.1 Objectifs

Dans ce chapitre, vous allez étudier plusieurs modèles dynamiques vus en cours :

1. modèle de mortalité exponentielle
2. modèle de croissance exponentielle
3. modèle de croissance logistique
4. modèle de Lotka-Volterra
5. modèles de compétition

Pour cela, dans `RStudio`, vous devrez construire ces modèles en utilisant des boucles `for` (dont nous détaillerons la syntaxe à la Section 3.3) afin de décrire les variations d'abondance des populations étudiées au cours du temps. Quand ces modèles seront construits, vous devrez en faire varier les conditions initiales et les valeurs des paramètres, afin d'observer le comportement du modèle dans des conditions variées.

Pour chaque modèle cité plus haut, vous allez donc tout d'abord simuler des données dans des conditions précises, en faire la représentation graphique, puis modifier les paramètres du modèle pour comprendre comment les différents termes (de croissance, de mortalité, d'interaction…) affectent les populations.

3.2 Pré-requis

Comme pour chaque nouveau chapitre, je vous conseille de travailler dans un nouveau script que vous placerez dans votre répertoire de travail, et dans une nouvelle session de travail (Menu `Session > Restart R`). Inutile en revanche de créer un nouveau `Rproject` : vous pouvez tout à fait avoir plusieurs scripts dans le même répertoire de travail et pour un même

`Rproject`. Comme toujours, consultez le livre en ligne du semestre 3 si vous ne savez plus comment faire.

Si vous êtes dans une nouvelle session de travail (ou que vous avez quitté puis relancé `RStudio`), vous devrez penser à recharger en mémoire les packages utiles. Dans ce chapitre, vous aurez uniquement besoin des packages du `tidyverse` (Wickham 2023), en particulier les packages `dplyr` (Wickham, François, et al. 2026), pour manipuler des tableaux, et `ggplot2` (Wickham, Chang, et al. 2026) pour les représentations graphiques.

```
library(tidyverse)
```

Nous n'aurons pas besoin ici d'importer de données depuis des fichiers externes : nous allons en effet générer directement des données à partir de différents modèles démographiques vus en cours.

Comme toujours, je spécifie une fois pour toutes le thème que j'utiliserai pour tous les graphiques de ce chapitre. Libre à vous de choisir un thème différent ou de vous contenter du thème proposé par défaut :

```
theme_set(theme_bw())
```

3.3 Les boucles `for`

Dans tous les langages de programmation, les boucles permettent de répéter automatiquement les mêmes actions un grand nombre de fois. Il existe plusieurs types de boucles, mais les boucles `for` sont les plus communes. Dans `R`, chaque boucle `for` possède 3 éléments :

1. **La sortie**. Avant d'exécuter la boucle, vous devez toujours créer de façon explicite un objet qui contiendra les résultats des calculs effectués à l'intérieur de la boucle. Une bonne façon de créer un vecteur vide pour stocker des valeurs numériques consiste à utiliser la fonction `numeric()`

2. **La séquence** pour laquelle vous souhaitez "faire tourner la boucle". Dans une boucle `for`, vous devez indiquer un indice qui spécifie le nombre d'itérations que la boucle doit effectuer. Par exemple :

   ```
   for (i in 1:10)
   ```

   indique que les commandes qui figurent à l'intérieur de la boucle devront être exécutées 10 fois, une fois pour chaque valeur de `i`, c'est-à-dire 1, 2, 3… jusqu'à 10. Pour chaque exécution des commandes de la boucle, l'indice `i` prendra donc une valeur différente comprise entre 1 et 10.
3. **Le corps de la boucle**. Il est constitué des commandes que l'on souhaite faire exécuter à la boucle, et il doit obligatoirement être encadré par des accolades `{ }`.

Voilà un exemple de boucle très simple :

```
# Création d'un vecteur numérique vide
res <- numeric()

# Pour chaque valeur de i comprise entre 1 et 5
for (i in 1:5) {
  res[i] <- 3 * i - 1 # Calcule 3 * i - 1
}

# Affichage des résultats
res
```

```
[1]  2  5  8 11 14
```

Les boucles peuvent faire des choses beaucoup plus complexes. Par exemple, supposez que vous avez une première valeur de 100, et que vous souhaitez diviser cette valeur par 2, puis que vous souhaitez diviser le résultat par 2, et ainsi de suite, 10 fois. Voilà comment faire ça avec une boucle `for` :

```
# Création d'un vecteur numérique vide
res <- numeric()
```

```r
# Stockage de la valeur initiale dans cet objet
res[1] <- 100

# Pour chaque valeur de i comprise entre 2 et 11
for (i in 2:11) {
  res[i] <- res[i - 1] / 2
}

# Affichage des résultats
res
```

```
 [1] 100.00000000  50.00000000  25.00000000  12.50000000   6.25000000
 [6]   3.12500000   1.56250000   0.78125000   0.39062500   0.19531250
[11]   0.09765625
```

Notez bien la position des accolades `{` et `}` dans le code ci-dessus. Leur position est importante, de même que les sauts de lignes. Notez aussi que dans cet exemple, `i` prend successivement les valeurs 2, 3, 4, … , 11, au lieu des valeurs 1, 2, 3, … , 10. En effet, on connait déjà la première valeur de `res`. On veut justement utiliser cette première valeur pour calculer la deuxième, la troisième et ainsi de suite. Si on souhaite répéter l'opération 10 fois de suite, il nous faut faire varier `i` de 2 à 11 inclus. Ici, nous avons en réalité calculé l'équation de récurrence suivante :

$$res_t = \frac{res_{t-1}}{2}$$

Vous savez maintenant comment fonctionnent les boucles `for` et vous devriez donc pouvoir construire votre premier modèle dynamique.

> **Attention**
>
> Dans `R`, le premier élément d'un objet occupe la position `[1]`. L'indice `[0]` n'existe pas. Il faut donc faire attention à ce que les indices choisis pour `i` dans l'expression `for (i in ...)` soient cohérents avec d'une part la position dans l'objet qui stockera les résultats, et d'autre part l'équation du système que l'on simule.

3.4 Modèle de mortalité exponentielle

Comme nous l'avons vu dans le cours, le modèle de mortalité exponentielle est défini par l'équation suivante :

$$\frac{\mathrm{d}N}{\mathrm{d}t} = -mN$$

Avec N, l'abondance de la population étudiée, t, le temps et m le taux de mortalité instantané. Pour coder ce modèle dans `RStudio`, vous devez vous rappeler de **la signification** de cette équation différentielle. Elle indique que dans un intervalle de temps très court dt, **la variation de l'abondance de la population** dN vaut $-mN$. Autrement dit, entre 2 temps d'observation successifs (par exemple entre t et $t+1$, ou entre $t-1$ et t), l'abondance de la population passe de N à $N-mN$. Ce modèle peut donc être décrit par l'équation de récurrence suivante :

$$N_{t+1} = N_t - m \times N_t$$

ce qui est évidemment équivalent à

$$N_t = N_{t-1} - m \times N_{t-1}$$

Cette dernière équation peut se lire ainsi : l'abondance au temps t (N_t) est égale à l'abondance au temps précédent N_{t-1}, plus la variation d'abondance entre 2 pas de temps successifs ($-m \times N_{t-1}$). Vous devez utiliser cette équation pour construire le modèle de mortalité exponentielle à l'intérieur d'une boucle `for` comme dans le dernier exemple de la Section 3.3. Ici, on cherche donc à déterminer la nouvelle valeur de la variable (N au temps t) en utilisant la valeur de la même variable mais au temps précédent (N au temps $t-1$).

Plusieurs étapes sont donc nécessaires :

1. Créez un vecteur `N` qui sera utilisé pour stocker les résultats des calculs à chaque étape de simulation. Utilisez la commande suivante :

```
# Création d'un vecteur numérique qui contiendra les résultats
N <- numeric()
```

2. Fixez l'abondance initiale de la population à $N_0 = 10$

3. Fixez le taux de mortalité instantanée à $m = 0.05$ (à chaque pas de temps, 5% de la population disparaît)
4. Placez l'abondance initiale à la première position de `N`
5. Créez une boucle `for` pour calculer et placer dans `N` l'abondance de la population pour 100 pas de temps
6. Créez un tibble nommé `res` contenant 2 variables (`Generation` et `Abondance`) en utilisant cette commande :

```
# Création d'un tibble où les résultats seront stockés
res <- tibble(Generation = 0:100, N)
```

Si votre script est correct et qu'il s'exécute sans erreur, l'objet `res` devrait contenir 101 lignes, avec 101 valeurs d'abondance. Les premières lignes de `res` devraient contenir ces valeurs :

```
# A tibble: 101 x 2
   Generation     N
        <int> <dbl>
 1          0 10
 2          1  9.5
 3          2  9.02
 4          3  8.57
 5          4  8.15
 6          5  7.74
 7          6  7.35
 8          7  6.98
 9          8  6.63
10          9  6.30
# i 91 more rows
```

Visualisez ces données avec `ggplot2`. Votre graphique devrait ressembler à ceci :

> Pour comprendre ce modèle…
>
> Essayez d'utiliser plusieurs valeurs différentes pour N_0 et relancez tout le script. Que se passe-t-il quand N_0 est très grand ? Est-ce normal ?
> Enfin, essayez de changer la valeur de m et de relancer le script. Faites des essais avec plusieurs valeurs. Qu'observe-t-on quand la valeur de m se rapproche de

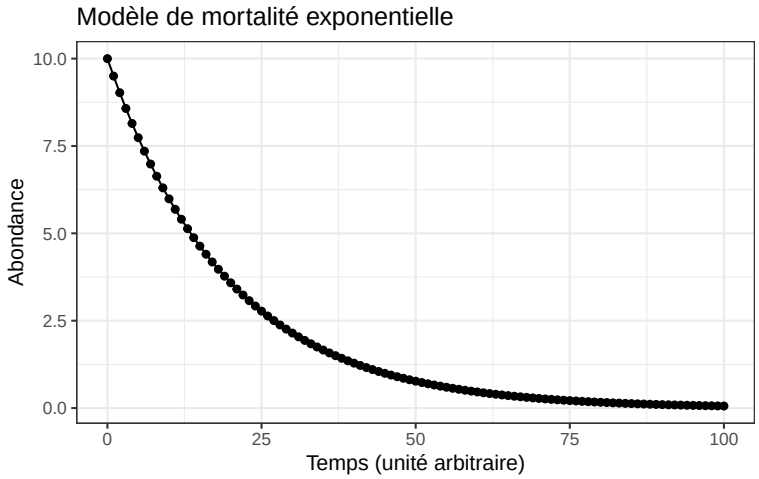


FIGURE 3.1 : Modèle de mortalité exponentielle avec $N_0 = 10$ et $m = 0.05$.

1 ? Est-ce normal ? Et que se passe-t-il si $m > 1$? Est-ce réaliste ? Pourquoi ? Mêmes questions avec $m = 2$ et $m = 3$?

3.5 Modèle de croissance exponentielle

Comme nous l'avons vu dans le cours, le modèle de croissance exponentielle est défini par l'équation suivante :

$$\frac{\mathrm{d}N}{\mathrm{d}t} = rN$$

Avec N, l'abondance de la population étudiée, t, le temps et r le taux de croissance instantané. Pour coder ce modèle dans `RStudio`, vous devez vous rappeler de **la signification** de cette équation différentielle. Elle indique que dans un intervalle de temps très court $\mathrm{d}t$, **la variation de l'abondance de la population** $\mathrm{d}N$ vaut $+rN$. Autrement dit, entre 2 temps d'observation successifs (par exemple entre t et $t+1$, ou entre $t-1$ et t), l'abondance de la population passe de N à $N+rN$. Ce modèle peut donc être décrit par l'équation de récurrence suivante :

$$N_{t+1} = N_t + r \times N_t$$

ce qui est évidemment équivalent à

$$N_t = N_{t-1} + r \times N_{t-1}$$

Cette dernière équation peut se lire ainsi : l'abondance au temps t (N_t) est égale à l'abondance au temps précédent N_{t-1}, plus la variation d'abondance entre 2 pas de temps successifs ($+r \times N_{t-1}$). Vous devez utiliser cette équation pour construire le modèle de mortalité exponentielle à l'intérieur d'une boucle `for` comme dans le dernier exemple de la Section 3.3. Ici, on cherche donc à déterminer la nouvelle valeur de la variable (N au temps t) en utilisant la valeur de la même variable mais au temps précédent (N au temps $t-1$).

Vous fixerez l'abondance initiale à $N_0 = 5$ et le taux de croissance instantané à $r = 0.03$. Vous calculerez l'abondance de la population pour 100 pas de temps. Vous devriez ainsi obtenir les résultats suivants :

```
# A tibble: 101 x 2
   Generation     N
        <int> <dbl>
 1          0  5
 2          1  5.15
 3          2  5.30
 4          3  5.46
 5          4  5.63
 6          5  5.80
 7          6  5.97
 8          7  6.15
 9          8  6.33
10          9  6.52
# i 91 more rows
```

Pour comprendre ce modèle…

Comme pour le modèle de mortalité exponentielle, essayez plusieurs valeurs pour la condition initiale N_0 et pour le paramètre du modèle r. Que se passe-t-il quand $r = 1$? Examinez les premières valeurs d'abondance dans le tableau `res` pour vous faire une meilleure idée. Est-ce réaliste ? Que se passe-t-il quand $r > 1$? Est-ce possible dans la nature ?

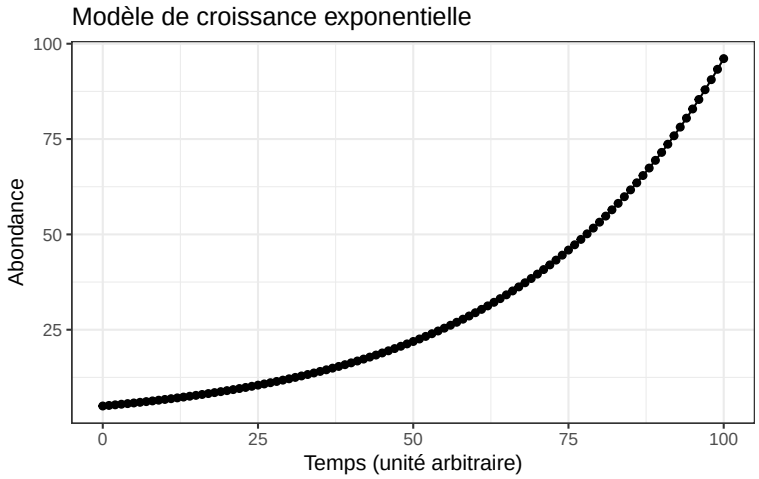


FIGURE 3.2 : Modèle de croissance exponentielle avec $N_0 = 5$ et $r = 0.03$.

3.6 Modèle de croissance logistique

À partir du modèle de croissance exponentielle construit à la Section 3.5, vous devez maintenant construire un modèle de croissance logistique qui est défini par l'équation suivante :

$$\frac{\mathrm{d}N}{\mathrm{d}t} = rN - \frac{r}{K} \times N^2$$

Avec N, l'abondance de la population étudiée, t, le temps, r le taux de croissance instantané, et K la capacité biotique (ou capacité de charge) du milieu. Comme toujours, commencez par écrire cette équation sous forme d'une relation de récurrence :

$$N_t = N_{t-1} + \cdots$$

Rappelez-vous que la nouvelle abondance (au temps t) est égale à l'ancienne (au temps $t-1$), plus la variation d'abondance entre 2 pas de temps successifs, la formule de la variation étant fournie par la formule $\frac{\mathrm{d}N}{\mathrm{d}t} = \cdots$

Vous devez donc modifier le script de la Section 3.5 pour ajouter le terme d'auto-limitation lié à l'environnement $\frac{r}{K} \times N^2$. Vous utiliserez les valeurs suivantes pour les conditions initiales et les paramètres du modèle :

— $N_0 = 1$
— $r = 0.05$
— $K = 80$

— durée de la simulation : 200 pas de temps

Vous devriez ainsi obtenir les résultats suivants :

```
# A tibble: 201 x 2
   Generation     N
        <int> <dbl>
 1          0  1
 2          1  1.05
 3          2  1.10
 4          3  1.16
 5          4  1.21
 6          5  1.27
 7          6  1.33
 8          7  1.40
 9          8  1.47
10          9  1.54
# i 191 more rows
```

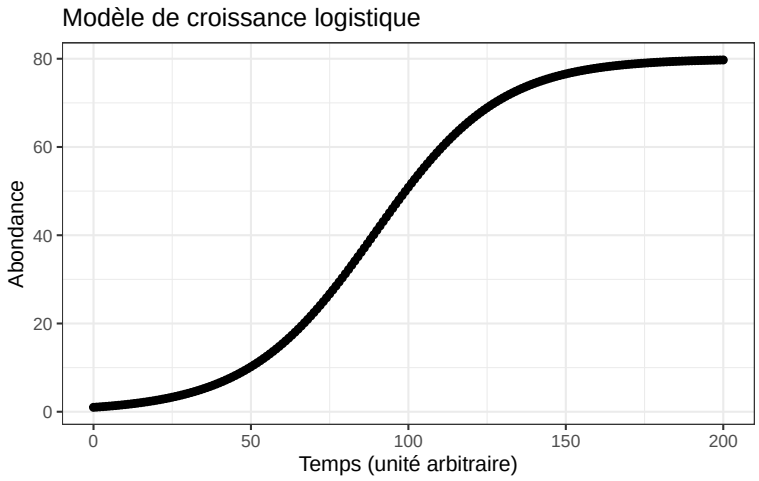


FIGURE 3.3 : Modèle de croissance logistique avec $N_0 = 1$, $r = 0.05$ et $K = 80$.

Pour comprendre ce modèle…

Modifiez les valeurs de N_0, r et K l'une après l'autre, pour comprendre comment chaque paramètre affecte la cinétique observée. Que se passe-t-il quand N_0 augmente ? Que se passe-t-il quand $N_0 > K$? Est-ce logique ? Pouvez-vous imaginer des situations réalistes où $N_0 > K$? Quelle est l'unité de K ?
Enfin, que se passe-t-il si le terme d'auto-limitation du

milieu est ajouté à la croissance exponentielle au lieu d'être soustrait ? Expliquez.

3.7 Modèle prédateur-proie de Lotka-Volterra

Vous allez maintenant étudier le modèle prédateur-proie tel que formulé par Lotka et Volterra et dont les équations vous ont été décrites dans le cours magistral de "fonctionnement des écosystèmes" :

$$\begin{cases} \dfrac{dX}{dt} &= rX - \frac{r}{K}X^2 - c_1XY \\ \dfrac{dY}{dt} &= -mY + c_2XY \end{cases}$$

avec X, l'abondance des proies, Y, l'abondance des prédateurs, r le taux de croissance instantanée des proies, K la capacité biotique du milieu, m le taux de mortalité instantanée des prédateurs, c_1 l'effet négatif de la prédation sur les proies (*i.e.* la mortalité des proies causée par les rencontres avec les prédateurs) et c_2 l'effet positif de la prédation sur les prédateurs (*i.e.* la croissance de la population de prédateurs causée par les rencontres avec les proies).

En utilisant la même démarche que précédemment, construisez un modèle de Lotka-Volterra avec les valeurs suivantes :

- Abondance initiale des proies : $X_0 = 1$
- Abondance initiale des prédateurs : $Y_0 = 10$
- Taux de croissance instantanée des proies : $r = 0.05$
- Taux de mortalité instantanée des prédateurs : $m = 0.05$
- Capacité biotique du milieu : $K = 50$
- Effet de la prédation sur les proies : $c_1 = 0.01$
- Effet de la prédation sur les prédateurs : $c_2 = 0.01$
- Durée de la simulation : 1000 pas de temps

Avec ce modèle, il vous faut calculer en parallèle l'abondance de 2 populations : celle des proies et celle des prédateurs.

! Calcul simultané

Attention : il est impossible de calculer d'abord toutes les abondances des proies, puis ensuite toutes les abondances des prédateurs, dans 2 boucles `for` distinctes. À chaque nouveau pas de temps, au sein d'une unique boucle `for`, il faut :

1. calculer la nouvelle abondance des proies à partir de l'abondance des proies et des prédateurs au temps précédent
2. calculer la nouvelle abondance des prédateurs à partir de l'abondance des proies et des prédateurs au temps précédent

Au final, l'objet `res` qui contiendra les résultats de vos simulations devra donc posséder 3 colonnes :

1. Une colonne `Generation` contenant le numéro des étapes de simulation (les pas de temps)
2. Une colonne `X` contenant les abondances de la population de proies à chaque étape de simulation
3. Une colonne `Y` contenant les abondances de la population de prédateurs à chaque étape de simulation

Avec ce tableau `res`, vous devriez pouvoir produire le diagramme des phases suivant :

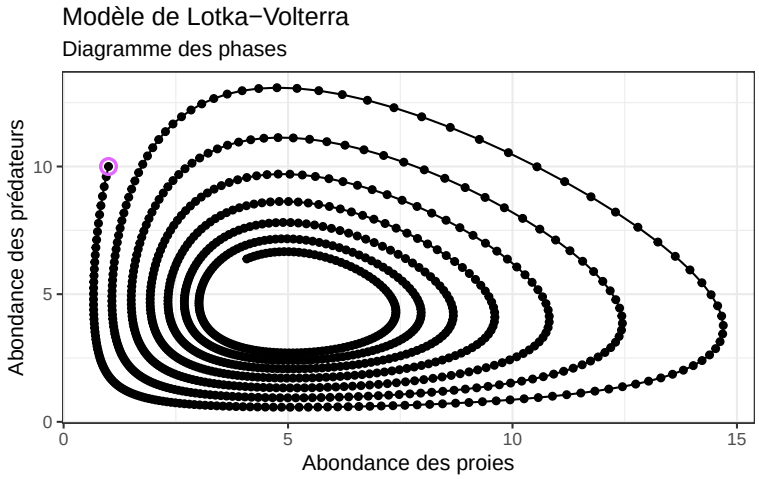
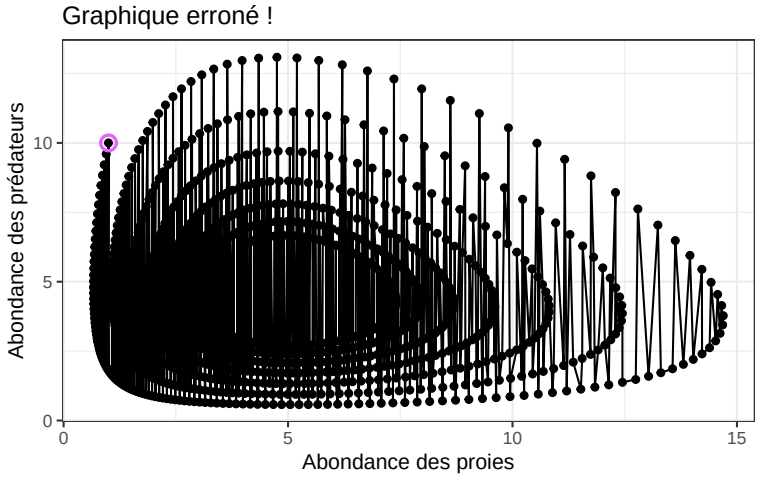


FIGURE 3.4 : Modèle de Lotka-Volterra avec $X_0 = 1$, $Y_0 = 10$, $r = 0.05$, $m = 0.05$, $K = 80$ et $c_1 = c_2 = 0.01$. Le cercle violet indique les conditions initiales.

Pour faire ce graphique, vous aurez besoin d'utiliser la fonction `geom_path()` au lieu de `geom_line()`. En effet, `geom_line()`

relie les points d'un graphique par ordre d'abscisse croissant :



Ça n'est pas ce que nous voulons obtenir ! On souhaite relier les points par ordre chronologique, donc dans l'ordre où ils apparaissent dans le tableau `res`.

Enfin, pour mieux visualiser la dynamique de ces deux populations au cours du temps, produisez le graphique suivant :

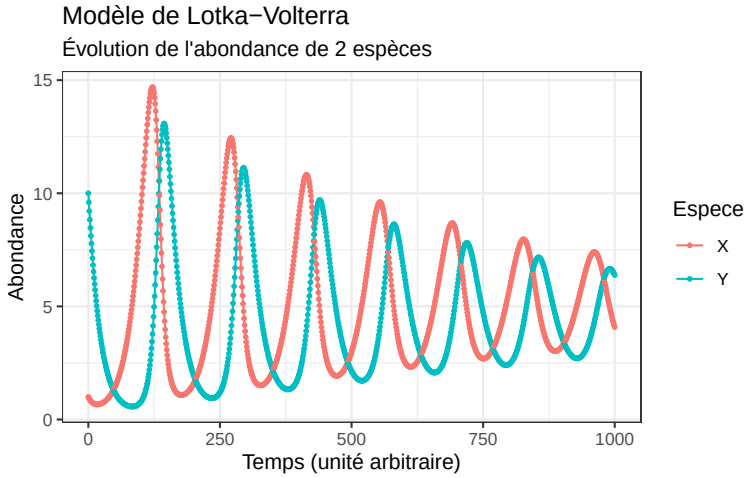


FIGURE 3.5 : Modèle de Lotka-Volterra avec $X_0 = 1$, $Y_0 = 10$, $r = 0.05$, $m = 0.05$, $K = 80$ et $c_1 = c_2 = 0.01$.

Pour produire ce graphique, vous aurez besoin de transformer le tableau `res` en format long. Une colonne `Espece` devra contenir soit `X`, soit `Y`, et une colonne `Abondance` devra contenir toutes les valeurs d'abondance des 2 espèces. Si vous ne savez plus comment faire, relisez le chapitre 1.3 du livre en

ligne du semestre 4. Voilà à quoi devraient ressembler les premières lignes de votre tableau :

```
res_long
```

```
# A tibble: 2,002 x 3
   Generation Espece Abondance
        <int> <chr>      <dbl>
 1          0 X          1
 2          0 Y         10
 3          1 X          0.949
 4          1 Y          9.6
 5          2 X          0.904
 6          2 Y          9.21
 7          3 X          0.866
 8          3 Y          8.83
 9          4 X          0.832
10          4 Y          8.47
# i 1,992 more rows
```

Pour comprendre ce modèle…

Expérimentez et tentez de répondre aux questions suivantes :

- Quelle sera l'issue de ce modèle prédateur-proie ? Augmentez la durée de simulation pour le prouver.
- Quelles seront les abondances des proies et des prédateurs quand l'équilibre sera atteint ?
- Modifiez la capacité biotique de la proie. Que se passe-t-il quand vous l'augmentez ? Que se passe-t-il quand vous la diminuez ? Est-ce normal ? Est-ce ce à quoi vous vous attendiez ?
- Que se passe-t-il quand vous modifiez les valeurs des autres paramètres du modèle (r, m, c_1 et c_2) ?
- Trouvez une combinaison de valeurs qui produisent un système à l'équilibre dynamique (*i.e.* un système d'oscillations harmoniques avec un diagramme des phases présentant un attracteur cyclique ou une courbe fermée)
- Est-ce que certaines combinaisons de paramètres produisent des résultats irréalistes ?

3.8 Modèle de compétition interspécifique

Dans sa forme la plus simple, on considère un système de compétition entre 2 espèces présentant un modèle de croissance logistique. La compétition ajoutera un terme de mortalité pour chaque espèce :

$$\begin{cases} \frac{dX}{dt} &= r_1 X - \frac{r_1}{K_1} X^2 - c_1 XY \\ \frac{dY}{dt} &= r_2 Y - \frac{r_2}{K_2} Y^2 - c_2 XY \end{cases}$$

avec X et Y l'abondance des deux espèces, r_1 et r_2 les taux de croissance instantanée des deux espèces, K_1 et K_2 les capacités biotiques pour les 2 espèces, et c_1 et c_2 les effets négatifs de la compétition sur chacune des 2 espèces (*i.e.* la mortalité causée par les rencontres entre les deux espèces en compétition).

En modifiant le système prédateur-proie précédent, codez ce système de compétition et essayez de trouver une combinaison de paramètres et de valeurs initiales permettant de reproduire les 4 situations décrites en cours :

1. Deux espèces très compétitives (avec différentes abondances initiales). Retrouvez-vous la ligne de catastrophe ?

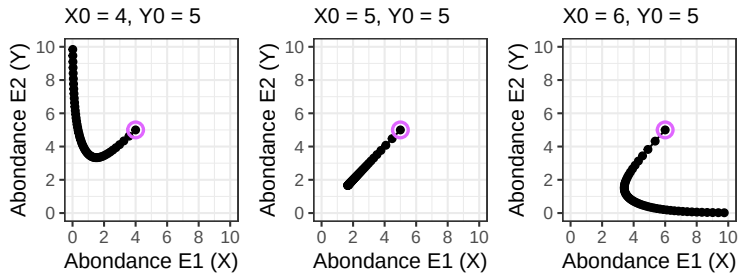


FIGURE 3.6 : Diagramme des phases de 2 espèces en compétition : 2 espèces très compétitives avec des conditions initiales différentes.

2. Deux espèces très peu compétitives
3. Première espèce plus compétitive que la seconde
4. Seconde espèce plus compétitive que la première

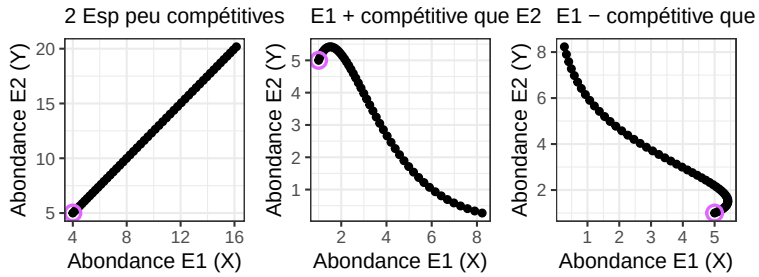


FIGURE 3.7 : Diagramme des phases de 2 espèces en compétition : 3 situations distinctes.

Pour comprendre ce modèle…

Attention à l'échelle des axes de vos graphiques. Pour bien comprendre ce qui se passe et retrouver les figures schématiques vues en cours, vous devez tenter de replacer les graphiques produits ici dans un plan plus large.
Notez aussi que contrairement au modèle prédateur-proie, les modèles de compétition ne font pas apparaître de cycles sur les diagrammes des phases. L'évolution des abondances au cours du temps ne fait donc jamais apparaître d'oscillations.

3.9 Modèles à 3 espèces

Sur la base des chapitres précédents, vous devriez maintenant pouvoir construire des modèles plus complexes, par exemple à 3 espèces en compétition, ou à 3 espèces dont 2 proies en compétition et une espèce de prédateur. La clé est ici de ne pas oublier de termes lorsque l'on construit le modèle : pour chaque espèce supplémentaire dans le modèle, la complexité augmente singulièrement. En effet, une espèce signifie une équation de plus dans le système, mais également des termes d'interaction supplémentaires dans les équations déjà existantes.

Par exemple, si on considère un système dans lequel 3 espèces sont présentes, deux espèces de proies en compétition et un prédateur, voilà à quoi ressemble le système d'équations :

$$\begin{cases} \frac{dX}{dt} &= r_1 X - \frac{r_1}{K_1} X^2 - c_{12} XY - c_{13} XZ \\ \frac{dY}{dt} &= r_2 Y - \frac{r_2}{K_2} Y^2 - c_{21} XY - c_{23} YZ \\ \frac{dZ}{dt} &= -mZ + c_{31} XZ + c_{32} YZ \end{cases}$$

Ce système de 3 équations comprend maintenant 11 paramètres : r_1, r_2, K_1, K_2, m, c_{12}, c_{21}, c_{13}, c_{23}, c_{31} et c_{32}.

Essayez de coder ce système et de produire une figure telle que la Figure 3.8.

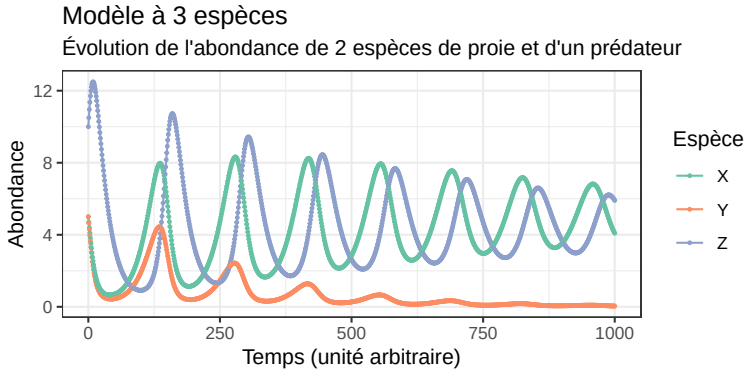


FIGURE 3.8 : Évolution de l'abondance de 2 espèces de proies en compétition et d'une espèce prédatrice. Ici, les 2 espèces de proies ont les mêmes caractéristiques démographiques et sont peu compétitives. Le prédateur a un impact plus important sur l'espèce de proie 2 (abondance Y) et conduit donc à son extinction.

Pour comprendre ce modèle…

Expérimentez et tentez de répondre aux questions suivantes :

- Pouvez-vous trouver une combinaison de paramètres conduisant à la coexistence des 3 espèces ?
- Pouvez-vous trouver une combinaison de paramètres conduisant à la coexistence de 2 des 3 espèces ?
- Que se passe-t-il si on augmente légèrement la capacité biotique de la première espèce ? À quelle espèce cela devrait profiter le plus ? Est-ce ce que l'on observe ? Cela vous paraît-il logique ?
- Dans le système d'équations ci-dessus, êtes-vous capables d'identifier à quoi correspond chacun des termes ? Quels sont les termes qui rendent compte de la compétition ? Quels sont les termes

qui rendent compte de la prédation ?

4 Les Chaînes de Markov

4.1 Pré-requis

Comme pour chaque nouveau chapitre, je vous conseille de travailler dans un nouveau script que vous placerez dans votre répertoire de travail, et dans une nouvelle session de travail (Menu `Session > Restart R`). Inutile en revanche de créer un nouveau `Rproject` : vous pouvez tout à fait avoir plusieurs scripts dans le même répertoire de travail et pour un même `Rproject`. Comme toujours, consultez le livre en ligne du semestre 3 si vous ne savez plus comment faire.

Si vous êtes dans une nouvelle session de travail (ou que vous avez quitté puis relancé `RStudio`), vous devrez penser à recharger en mémoire les packages utiles. Dans ce chapitre, vous aurez uniquement besoin des packages du `tidyverse` (Wickham 2023), en particulier les packages `dplyr` (Wickham, François, et al. 2026), pour manipuler des tableaux, et `ggplot2` (Wickham, Chang, et al. 2026) pour les représentations graphiques. Vous pourrez aussi avoir besoin du package `scales` (Wickham et al. 2025) pour améliorer la mise en forme des axes des graphiques produits

```
library(tidyverse)
library(scales)
```

Nous n'aurons pas besoin ici d'importer de données depuis des fichiers externes : nous allons en effet générer directement des données à partir d'une chaîne de Markov.

Comme toujours, je spécifie une fois pour toutes le thème que j'utiliserai pour tous les graphiques de ce chapitre. Libre à vous de choisir un thème différent ou de vous contenter du thème proposé par défaut :

```
theme_set(theme_bw())
```

4.2 Les matrices dans R

Lors de votre découverte du logiciel R, de son fonctionnement et de sa syntaxe, vous avez appris qu'il existe plusieurs types d'objets que l'on peut créer et manipuler : des vecteurs, des facteurs, des data.frames ou tibbles, des listes, etc. L'un des types d'objets que nous n'avons pas encore décrit est la matrice.

Dans R, une matrice est un tableau dans lequel chaque cellule contient des éléments du même type. Contrairement aux data.frames et aux tibbles, une matrice ne peut donc pas avoir des chiffres dans une colonne, et des chaînes de caractères ou des vrais-faux dans une autre. De ce point de vue, une matrice est donc similaire à un vecteur qui ne peut contenir que des éléments qui sont tous du même type. Mais comme un data.frame ou un tibble, et contrairement à un vecteur, une matrice possède des dimensions : un nombre de lignes et un nombre de colonnes.

Les matrices sont donc des objets qui possèdent des points communs et des différences, avec les vecteurs et avec les data.frames ou tibbles.

4.2.1 Création

La façon la plus simple de créer une matrice est d'utiliser la fonction `matrix()`. Cette fonction possède de nombreux arguments et vous devrez au moins renseigner les suivants :

- `data` : quelles données souhaitez-vous placer dans la matrice. Il peut s'agir d'une unique valeur si vous souhaitez que toutes les cellules de la matrice contiennent la même chose. En général, il s'agit d'un vecteur contenant les éléments que l'on souhaite placer dans la matrice.
- `nrow` : combien de lignes la matrice que vous souhaitez créer devra-t-elle contenir ?
- `ncol` : combien de colonnes la matrice que vous souhaitez créer devra-t-elle contenir ?
- `byrow` : souhaitez-vous que les données fournies dans `data` soient remplies en lignes (`byrow = TRUE`) ou en colonnes (`byrow = FALSE`, c'est le choix par défaut) dans la matrice ?

En général, on renseigne donc systématiquement `data`, et soit `nrow`, soit `ncol`. Il n'est pas nécessaire de renseigner à la fois `nrow` et `ncol` puisque `R` est capable de déterminer l'un des deux automatiquement selon la taille du vecteur que vous avez fourni à `data`. Par exemple, si vous fournissez un vecteur de 15 éléments à data, et que vous indiquez `nrow = 3`, `R` produira automatiquement une matrice de 3 lignes et 5 colonnes. Cela suppose évidemment que la longueur du vecteur fourni soit compatible avec les dimensions de la matrice que vous envisagez.

Vous pouvez évidemment spécifier explicitement le nombre de lignes et le nombre de colonnes souhaitées, mais alors la longueur du vecteur fourni devra être parfaitement compatible avec les dimensions de la matrice envisagée. Par exemple, si vous souhaitez une matrice de 4 lignes et 4 colonnes (soit 16 cellules), vous devrez fournir exactement 16 valeurs dans le vecteur transmis à l'argument `data`. Si vous en fournissez plus ou moins, des choses inattendues peuvent se produire :

- si vous fournissez trop de valeurs, les valeurs en trop ne seront pas intégrées à la matrice, votre vecteur sera donc tronqué après la seizième valeur
- si vous fournissez trop peu de valeurs, le recyclage sera utilisé pour combler la matrice qui ne peut pas avoir de cellule vide : les premières valeurs du vecteur seront réutilisées jusqu'à ce que la matrice soit remplie, et un message d'avertissement vous sera renvoyé.

Voici quelques exemples :

```
# Création d'un vecteur de 8 éléments
a <- c(1, 5, 6, 9, 10, 5, 4, 2)  # 8 éléments

# Création d'une matrice de 2 lignes et 4 colonnes
matrix(data = a, nrow = 2)
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    6   10    4
[2,]    5    9    5    2
```

```
# Une autre façon de créer la même matrice
matrix(data = a, ncol = 4)
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    6   10    4
[2,]    5    9    5    2
```

Si la longueur du vecteur contenant les données n'est pas compatible avec les dimensions voulues pour la matrice, il faut spécifier les nombres de lignes et de colonnes de façon explicite, et un message d'avertissement est renvoyé.

```
# Pour créer une matrice de 3 lignes et 4 colonnes, il faut spécifier
# explicitement nrow et ncol
matrix(data = a, nrow = 3, ncol = 4)
```

```
Warning in matrix(data = a, nrow = 3, ncol = 4): data length [8] is not a
sub-multiple or multiple of the number of rows [3]
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    9    4    5
[2,]    5   10    2    6
[3,]    6    5    1    9
```

Ici, les premières valeurs du vecteur `a` ont été recyclées (1, 5, 6 et 9) pour remplir toutes les cellules de la matrice.

Dans tous les exemples ci-dessus, les chiffres contenus dans le vecteur `a` ont été remplis en colonnes dans la matrice. Pour remplir la matrice en lignes, on indique `byrow = TRUE` :

```
matrix(data = a, nrow = 2, byrow = TRUE)
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    5    6    9
[2,]   10    5    4    2
```

```
matrix(data = a, ncol = 4, byrow = TRUE)
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    5    6    9
[2,]   10    5    4    2
```

```r
matrix(data = a, nrow = 3, ncol = 4, byrow = TRUE)
```

```
Warning in matrix(data = a, nrow = 3, ncol = 4, byrow = TRUE): data length [8]
is not a sub-multiple or multiple of the number of rows [3]
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    5    6    9
[2,]   10    5    4    2
[3,]    1    5    6    9
```

Et bien sûr, comme vous le savez maintenant, si vous souhaitez pouvoir réutiliser les matrices créées avec la fonction `matrix()`, vous devez leur donner un nom avec l'opérateur `<-` :

```r
mat <- matrix(data = a, nrow = 3, ncol = 4, byrow = TRUE)
```

```
Warning in matrix(data = a, nrow = 3, ncol = 4, byrow = TRUE): data length [8]
is not a sub-multiple or multiple of the number of rows [3]
```

```r
mat
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    5    6    9
[2,]   10    5    4    2
[3,]    1    5    6    9
```

4.2.2 Produit matriciel

Le produit matriciel est l'opération qui permet de multiplier 2 matrices entre elles, ou de multiplier une matrice par un vecteur. Comme nous l'avons vu en cours, les dimensions des objets que l'on multiplie sont importantes.

! Produit matriciel

Si A et B sont deux matrices, le produit matriciel $A \times B$ n'est possible que si le nombre de colonnes de A est égal au nombre de lignes de B.
Par ailleurs, sauf cas particulier, $A \times B \neq B \times A$

Dans R, l'opérateur qui permet d'effectuer un produit matriciel est %*%.

```r
# Création d'une matrice A
A <- matrix(1:15, nrow = 5)
A
```

```
     [,1] [,2] [,3]
[1,]    1    6   11
[2,]    2    7   12
[3,]    3    8   13
[4,]    4    9   14
[5,]    5   10   15
```

```r
dim(A)
```

```
[1] 5 3
```

```r
# Création d'une matrice B
B <- matrix(1:15, ncol = 5)
B
```

```
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    4    7   10   13
[2,]    2    5    8   11   14
[3,]    3    6    9   12   15
```

```r
dim(B)
```

```
[1] 3 5
```

```r
# Produit matriciel A * B
A %*% B
```

```
     [,1] [,2] [,3] [,4] [,5]
[1,]   46  100  154  208  262
[2,]   52  115  178  241  304
[3,]   58  130  202  274  346
[4,]   64  145  226  307  388
[5,]   70  160  250  340  430
```

```r
# Produit matriciel B * A
B %*% A
```

```
     [,1] [,2] [,3]
[1,]  135  310  485
[2,]  150  350  550
[3,]  165  390  615
```

Vous constatez ici que les dimensions de la matrice obtenue sont variables et dépendent du sens du produit matriciel et des dimensions des matrices multipliées. Multiplier 2 matrices dont les dimensions sont incompatibles fait apparaître un message d'erreur :

```r
# Création d'une matrice C (4 lignes et 3 colonnes)
C <- matrix(1:12, ncol = 3)
C
```

```
     [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
[3,]    3    7   11
[4,]    4    8   12
```

```r
# Affichage de la matrice A (5 lignes et 3 colonnes)
A
```

```
     [,1] [,2] [,3]
[1,]    1    6   11
[2,]    2    7   12
[3,]    3    8   13
[4,]    4    9   14
[5,]    5   10   15
```

```r
# Calcul des produits matriciels : impossibles !
A %*% C
```

```
Error in A %*% C: non-conformable arguments
```

```
C %*% A
```

```
Error in C %*% A: non-conformable arguments
```

Enfin, il est tout à fait possible d'effectuer un produit matriciel entre un vecteur et une matrice. Dans `R`, les vecteurs n'ont pas de dimension, mais ils ont une longueur (*i.e.* un nombre d'éléments), que l'on peut afficher avec la fonction `length()`. Lorsque l'on effectue un produit matriciel impliquant un vecteur, le vecteur est automatiquement transformé soit en vecteur ligne (une matrice d'une seule ligne) soit en vecteur colonne (une matrice d'une seule colonne, voir le cours de "Fonctionnement des écosystèmes") :

- Si le vecteur est à gauche (`vec %*% mat`), il est transformé en vecteur ligne. Pour que le produit matriciel puisse être calculé, il faut alors que le nombre d'éléments de `vec` soit égal au nombre de lignes de `mat`.
- Si le vecteur est à droite (`mat %*% vec`), il est transformé en vecteur colonne. Pour que le produit matriciel puisse être calculé, il faut alors que le nombre d'éléments de `vec` soit égal au nombre de colonnes de `mat`.

```
# Création d'un vecteur
vec <- c(2, 4, 3)
vec
```

```
[1] 2 4 3
```

```
# Création d'une matrice D : 3 lignes et 2 colonnes
D <- matrix(c(5, 6, 8, 4, 2, 3), nrow = 3)
D
```

```
     [,1] [,2]
[1,]    5    4
[2,]    6    2
[3,]    8    3
```

```r
# Création d'une matrice E : 2 lignes et 3 colonnes
E <- matrix(c(1, 2, 3, 9, 8, 7), nrow = 2)
E
```

```
     [,1] [,2] [,3]
[1,]    1    3    8
[2,]    2    9    7
```

```r
# Calcul de différents produits matriciels
vec %*% D
```

```
     [,1] [,2]
[1,]   58   25
```

```r
D %*% vec
```

```
Error in D %*% vec: non-conformable arguments
```

```r
E %*% vec
```

```
     [,1]
[1,]   38
[2,]   61
```

```r
vec %*% E
```

```
Error in vec %*% E: non-conformable arguments
```

Là encore, les dimensions de la matrice obtenue à l'issue du produit matriciel dépendent des dimensions des objets multipliés et de l'ordre de la multiplication.

4.3 Simulation d'écosystème

Nous avons vu dans le cours magistral de "Fonctionnement des écosystèmes" qu'il était possible de simuler l'évolution de plusieurs types de communautés végétales en utilisant une chaîne de Markov très simple. Il nous faut pour cela 2 objets :

1. une matrice de transition P : il s'agit d'une matrice carrée qui se lit en lignes (la somme de chaque ligne vaut 1) et qui contient les probabilités de transition d'un type de communauté végétale à un autre entre 2 observations successives de l'écosystème étudié (tous ses coefficients sont donc soit nuls, soit strictement positifs et inférieurs à 1).
2. un vecteur π_0 qui contient les proportions de chaque type d'écosystème au temps zéro (il s'agit donc des conditions initiales)

Pour connaître la situation de l'écosystème au temps 1 ($\pi 1$), il suffit d'effectuer un produit matriciel :

$$\pi_1 = \pi_0 \times P$$

Pour connaître la situation de l'écosystème au temps 2 (π_2), il suffit d'effectuer un produit matriciel :

$$\pi_2 = \pi_1 \times P = \pi_0 \times P \times P$$

Pour connaitre la situation de l'écosystème au temps t (π_t), il suffit d'effectuer un produit matriciel :

$$\pi_t = \pi_{t-1} \times P = \pi_0 \times P^t$$

En utilisant ce que vous venez d'apprendre sur les matrices d'une part (création et produits matriciels) et ce que vous avez appris sur les boucles `for` dans la Section 3.3, simulez l'évolution de l'écosystème décrit en cours. Vous prendrez soin de respecter l'ordre des étapes suivant :

1. Création de la matrice de transition `P`
2. Création du vecteur des conditions initiales `pi0`

3. Réglage de la durée de simulation à 50 pas de temps
4. Création d'une matrice remplie de 0 qui accueillera les résultats des calculs pour chaque étape de simulation
5. Placer `pi0` sur la première ligne de la matrice
6. Calcul des 50 produits matriciels à l'aide d'une boucle `for`
7. Transformation de la matrice de résultat en tibble grâce à la fonction `as_tibble()`. Attention, n'oubliez pas d'ajouter une colonne `Temps` pour indiquer le numéro des étapes de simulation (de 0 à 50)
8. Transformation du tibble au format long avec `pivot_longer()`
9. Représentation graphique des résultats obtenus avec `ggplot()`.

Pour mémoire, voilà les données utilisées dans le cours pour simuler l'écosystème méditerranéen d'intérêt et ses 5 communautés végétales.

— Conditions initiales `pi0` :

```
pi0
```

```
  Chêne   Vigne Pelouse Garigue  Pinède
   0.20    0.35    0.10    0.15    0.20
```

— Matrice de transition `P` :

```
P
```

```
         Chêne Vigne Pelouse Garigue Pinède
Chêne      0.8   0.2    0.00     0.0   0.00
Vigne      0.0   0.7    0.30     0.0   0.00
Pelouse    0.0   0.0    0.40     0.6   0.00
Garigue    0.0   0.0    0.00     0.2   0.80
Pinède     0.1   0.0    0.25     0.0   0.65
```

Le graphique que vous devriez être en mesure de produire est le suivant :

🔥 Pour comprendre ce système…

— Essayez de modifier les conditions initiales en choisissant d'autres valeurs pour `pi0`. Attention, tous les coefficients représentent des proportions. Ils

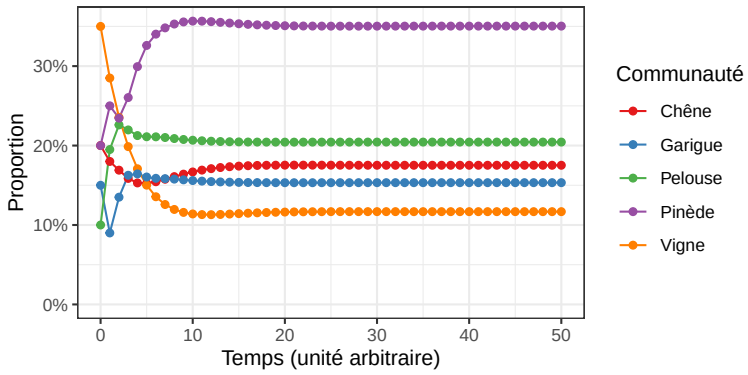


FIGURE 4.1 : Évolution des proportions de 5 types de communautés végétales au sein d'un écosystème méditerranéen (50 pas de temps)

doivent donc obligatoirement être compris entre 0 et 1 et leur somme doit faire 1.

— Que se passe-t-il si les forêts de chênes occupent 100% de l'écosystème au temps 0 ? Est-ce que la situation d'équilibre est modifiée ? Cela vous semble-t-il logique ? Essayez de l'expliquer en revenant au cours et en examinant les conditions requises pour qu'un équilibre stable apparaisse. Est-ce que cela dépend de π_0 ?

— Reprenez les valeurs de départ pour `pi0`, mais changez maintenant les probabilités de transition de la matrice `P`. Attention, là encore, tous les coefficients doivent être compris entre 0 et 1 et la somme de chaque ligne doit faire exactement 1 (P est une matrice stochastique).

— Est-ce que l'état d'équilibre est modifié ? Est-ce logique ?

— En tant que gestionnaire de l'environnement, votre objectif est maintenant d'arriver à une situation où les forêts de chênes sont plus abondantes que les pinèdes dans cet écosystème. Quels sont vos leviers d'action ? Pouvez-vous trouver des coefficients pour `P` qui reflètent ces changements ? À l'équilibre, cela se traduit-il par une dominance des forêts de chênes ?

Références

Macdonald, Peter, et with contributions from Juan Du. 2018. *mixdist : Finite Mixture Distribution Models*. https://www.r-project.org/.

Wickham, Hadley. 2023. *tidyverse : Easily Install and Load the Tidyverse*. https://tidyverse.tidyverse.org.

Wickham, Hadley, Mara Averick, Jennifer Bryan, et al. 2019. « Welcome to the tidyverse ». *Journal of Open Source Software* 4 (43) : 1686. https://doi.org/10.21105/joss.01686.

Wickham, Hadley, Winston Chang, Lionel Henry, et al. 2026. *ggplot2 : Create Elegant Data Visualisations Using the Grammar of Graphics*. https://ggplot2.tidyverse.org.

Wickham, Hadley, Romain François, Lionel Henry, Kirill Müller, et Davis Vaughan. 2026. *dplyr : A Grammar of Data Manipulation*. https://dplyr.tidyverse.org.

Wickham, Hadley, Thomas Lin Pedersen, et Dana Seidel. 2025. *scales : Scale Functions for Visualization*. https://scales.r-lib.org.